Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)
Faculty of Computer Science and Business Information Systems

Master Thesis

Great Research About My Favourite Topic

submitted for the completion of degree

Master of Science

in

Artificial Intelligence

Your Name

Submitted on: March 30, 2026

Supervisor: Prof. Dr. Magda Gregorova
Second Examiner: Prof. Dr. C D

Abstract (en)

A single paragraph summarizing the most important points of your thesis. This can be more than one paragraph but should not be too long. You usually write this as the very last thing of your thesis.

Abstract (de)

We are in germany so include a translation.

Acknowledgment

Here you can thank your friends who helped you throughout the studies, your parents for supporting you generally, if you wish your supervisor or any other profs and generally everybody you value.

Contents

1 Introduction

Vector graphics differ fundamentally from raster images. Raster images represent visual content as grids of pixels, where each pixel stores color information independently of its neighbors. Vector graphics instead describe images using mathematical primitives: lines, curves, shapes, and groups defined parametrically. Because the geometry is encoded rather than sampled, vector graphics scale without loss and can be edited at the level of individual components.

Among vector formats, Scalable Vector Graphics (SVG) has become the dominant web standard. SVG is a text-based format that encodes geometry, styling, and hierarchical relationships between elements. Its structured nature makes it programmable: SVG files can be generated, modified, and animated through code, which is why the format underlies most of what is drawn on the web. A 2024 survey by W3Techs found that over 65% of all websites use SVG for their graphics.

From the perspective of generative modeling, this structure introduces challenges that do not arise in raster synthesis. SVG graphics explicitly encode geometry and topology. A generated SVG must therefore satisfy two requirements simultaneously: it must look correct, and it must be structurally sound. An output that contains redundant paths, broken hierarchy, or unstable geometry may appear visually plausible while being impossible to edit or reuse. Visual quality and structural quality are distinct properties, and current evaluation practice does not consistently distinguish between them.

Before learning-based approaches, vector graphics were produced through algorithmic vectorization: tools that approximate raster images using geometric primitives via edge detection, contour tracing, and polygon approximation. These methods remain in widespread use but rely on heuristic rules and struggle with complex scenes, stylized content, or semantic guidance. The emergence of learning-based approaches introduces new possibilities, but also raises questions about how models construct vector structure internally and whether the outputs they produce are usable in practice.

1.1 Motivation

In recent years, generative models have substantially changed what is possible in visual content creation. Diffusion models, large-scale multimodal systems, and transformer-based architectures have demonstrated that high-quality image synthesis from text prompts is no longer a research prototype. Systems such as Stable Diffusion, DALL-E, and Midjourney produce raster images that are often indistinguishable from photographs or hand-drawn illustrations. Progress in this area has been fast and sustained.

This progress has not extended equally to vector graphics. Research on SVG generation has grown, but the reliability, controllability, and structural quality of generated outputs remain open problems. At the same time, vector graphics have not become less important. Icons, logos, interface elements, illustrations, and data visualizations are created and maintained in vector formats precisely because they must remain editable and resolution-independent. The formats used by Adobe Illustrator, Figma, and every major web browser are vector-based.

The gap between raster and vector generation is not simply a matter of technical difficulty. It reflects a deeper mismatch between how generative models are trained and evaluated and what vector graphics actually require. A model trained to minimize pixel reconstruction error has no reason to produce clean paths or interpretable layers. A model evaluated by FID score is rewarded for visual plausibility, not structural usability. Closing this gap requires understanding what current methods actually produce at the structural level, and why their outputs differ.

This is the motivation for the present work. Rather than proposing a new generation model, this thesis analyzes existing methods from the inside: how they represent vector graphics, what supervision signals they use, and how those choices shape the structure of the outputs they generate.

1.2 Research Problem

SVG generation research has expanded across several directions. Sequence-based models treat SVG as a stream of drawing commands. Optimization-driven approaches refine vector parameters directly against a target. Neural implicit representations define geometry through learned functions. Multimodal pipelines combine text encoders, diffusion models, and differentiable rasterizers. These methods differ not only in architecture but in how they internally represent vector structure and what signals they use to guide generation.

Despite this diversity, a systematic understanding of how these design choices influence structural outcomes is missing. Much of the literature reports visual similarity metrics while leaving structural properties (path count, layer organization, geometric stability, editability) unmeasured. This means that two models may achieve similar FID scores while producing outputs with very different structural properties. Comparison between approaches is difficult because reported results are heterogeneous, and the metrics used do not capture the properties that matter most for practical use.

A further difficulty is that evaluation practices for SVG generation remain inconsistent. Quantitative metrics such as FID and LPIPS are adapted from raster synthesis and measure visual similarity, not structural quality. There is no widely adopted benchmark for SVG editability, path economy, or layer coherence. Consequently, the field currently lacks the tools needed to systematically compare structural outcomes across generation approaches.

This combination (heterogeneous architectures, unmeasured structural properties, and inconsistent evaluation) constitutes the research gap addressed by this thesis.

1.3 Thesis Objective

This thesis investigates SVG generation methods from the perspective of representation, supervision, and structural quality. The goal is not to propose a new model but to develop a structured analytical understanding of how existing methods construct vector graphics and why their structural outcomes differ.

The analysis focuses on models where the generation process is interpretable in terms of vector operations or clearly defined architectural components. Large language model approaches, which treat SVG generation as code synthesis, are reviewed as a distinct and growing direction but excluded from the main analysis. Their generation process is distributed across billions of parameters rather than explicitly tied to geometric operations, which limits the kind of representation-level analysis this thesis pursues. A full justification is provided in Appendix **??**.

To support systematic comparison, the analysis is restricted to open-source models with available implementations and published architectural descriptions. This enables a study of methods that differ meaningfully in representation strategy, supervision signal, and optimization approach, while remaining grounded in verifiable technical detail.

1.4 Research Goals

1. To explain and analyze the core properties, structures, and components of SVG graphics as a foundation for understanding their representation and generation.

2. To collect, review, and categorize existing open-source methods for SVG generation across different input modalities.

3. To investigate the deep learning architectures employed in SVG generation models and analyze their design principles.

4. To examine how these architectures represent SVG data internally and how outputs are transformed into standard SVG files.

5. To evaluate existing approaches, metrics, and benchmarks for assessing the quality of generated SVGs.

6. To assess and compare the structural and visual quality of current SVG generation models based on identified evaluation criteria.

1.5 Research Questions

1. What are the main properties and structures that define an editable and high-quality SVG?

2. What open-source methods currently exist for generating SVGs from different input modalities?

3. What types of deep learning architectures are used in SVG generation models?

4. What kinds of structure do these models produce?

5. How can the quality and editability of generated SVG be evaluated?

6. What is the observed quality of SVG outputs across different models?

These questions are not independent. Together they form a single analytical inquiry: how do the internal design choices of SVG generation models (what they represent, how they are supervised, and how their outputs are evaluated) determine whether the resulting graphics are structurally usable as well as visually plausible. The thesis argues that

this question cannot be answered by visual metrics alone, and that systematic analysis of representation and supervision is necessary to explain the structural differences observed across current methods.

1.6 Thesis Structure

The background chapter introduces the foundational concepts required to understand vector graphics generation: SVG structure, representation types, differentiable rendering, and the supervision strategies used in learning-based methods.

Building on this, Chapter 4 surveys existing SVG generation approaches and categorizes them by architectural family and input modality. Chapters 5 and 6 analyze how representation and supervision choices shape vector structure, tracing the relationship between internal design decisions and observable output properties.

Chapter 7 introduces a multi-dimensional evaluation framework for SVG generation that extends beyond visual similarity to include structural and practical criteria. These perspectives are then applied in Chapter 8, where strengths, limitations, and trade-offs across the analyzed models are examined comparatively.

The thesis concludes by mapping findings to the research questions, summarizing contributions, and identifying directions for future work.

2 Systematic Literature Review

Kitchenham's guidelines were originally developed for evidence-based software engineering and have become one of the most widely adopted standards for conducting systematic literature reviews in computer science and technical research disciplines [**kitchenham˙procedures˙2004**]. The methodology provides detailed, domain-specific procedures for planning, searching, selecting, and synthesizing studies in areas where research outputs are often heterogeneous—such as algorithmic models, machine learning architectures, or engineering pipelines. Unlike biomedical-focused protocols such as PRISMA [**moher˙preferred˙2009**], which were designed primarily for clinical trials and human-subject research with standardized quantitative outcomes, Kitchenham addresses challenges unique to computational research: diversity of study designs, inconsistent terminology, non-uniform evaluation metrics, and the frequent presence of code-based or algorithmic artifacts instead of comparable statistical datasets. Similarly, frameworks like SALSA [**noauthor˙typology˙nodate**] emphasize high-level structuring but do not prescribe the step-by-step operational procedures needed for reproducibility in technical fields.

For this thesis—situated within deep learning, machine learning, and vector-graphics generation—a Kitchenham-style SLR is needed because the literature consists of diverse model architectures, varied input modalities, and different evaluation criteria. Kitchenham's protocol supports this complexity by providing structured phases (planning, conducting, and reporting) that guide the reviewer through formulating rigorous research questions, defining search strategies, conducting selection criteria, assessing quality, data extraction and performing narrative synthesis. This structure minimizes bias, increases reproducibility, and ensures that the review captures the full range of relevant deep-learning and vectorization approaches.

2.1 Phase 1 — Planning

2.1.1 Review Protocol

Databases

- arXiv
- Google Scholar
- ACM Digital Library

Search Strings/Keywords

SVG Generation

```
"SVG generation"
"scalable vector graphics" AND generation
"SVG generative models"
"vector graphics generation"
"SVG generation" AND deep learning
"SVG generation" AND machine learning
```

Raster-to-Vector Vectorization

```
"raster-to-vector" AND deep learning
"image-to-vector" AND neural network
"image vectorization" AND machine learning
"deep vectorization"
"differentiable vector graphics"
```

Text-to-SVG

```
"text-to-SVG"
"text-to-vector graphics"
```

Structure of SVG, Editability and Quality

```
"SVG Basics"
"SVG fundamentals"
"Scalable Vector Graphics"
```

```
"SVG editability"
"editable SVG"
"SVG path simplification"
"layered SVG" AND generation"
```

Survey and Review Searches

```
"SVG" AND "systematic literature review"
"vector graphics" AND survey
"vector image" AND review
"image vectorization" AND survey
"graphic generation" AND review
"SVG Generation" AND benchmark
"image vectorization" AND benchmark
```

Study Selection Criteria

Inclusion Criteria:

- **Language and Publication:** Written in English and published in a peer-reviewed journal, conference, or workshop.
- **Time Range:** Published between 2020–2025 (post-emergence of modern SVG generative models such as DeepSVG).
- **Domain Relevance:** Falls within computer vision, computer graphics, AI, machine learning, or deep learning.
- **Reproducibility:** Provides an open-source implementation or references a publicly available repository. Availability of pre-trained weights is preferred, but source code alone is sufficient if the method is reproducible.
- **SVG Output Requirement:** Describes a model, method, or architecture that directly generates SVG or structured vector outputs, including:
  - Bézier curves,
  - SVG `<path>` commands,
  - layered or hierarchical SVG structure,

  - vector strokes or SVG-like stroke sequences,
  - structured outputs suitable for editability analysis.
- **Input Modalities:** Supports at least one of:
  - image-to-SVG vectorization,
  - text-to-SVG generation,
  - multimodal SVG generation,
- **Architectural Transparency:** Provides sufficient technical detail to extract the model architecture and internal SVG representation (e.g., decoder design, transformer layers, vector tokenization, differentiable rasterization, or geometric parameterization).
- **Evaluation:** Provides quantitative metrics and qualitative SVG examples that allow analysis of structural quality and editability.

Exclusion Criteria:

- **No Vector Output:** The method does not generate vector or SVG outputs (pixel-only image generation).
- **Non-ML Methods:** Purely rule-based or classical algorithmic vectorization methods without machine learning or deep learning.
- **Closed-Source Systems:** No publicly available code, weights or sufficient technical detail for reproduction.
- **Insufficient Evaluation:** Papers lacking SVG evaluation, qualitative vector examples or structural analysis of outputs.
- **Out-of-Scope Vector Work:** Studies focused on vector editing, rendering, compression, markup optimization, or stylistic modification without SVG generation.
- **Domain Mismatch:**
  - handwriting synthesis,
  - font generation,

  - datasets,
  - sketch generation not intended for SVG graphics.

- **Temporal Exclusion:** Studies published before 2020, except when cited for background context only.
- **LLM Restriction:** Exclude proprietary LLMs that generate SVG text but do not provide architecture, training details, or reproducibility.

Study Selection Procedure

- Initial records
- Duplicate removal
- Title/abstract screening
- Full-text assessment
- Final included studies

Quality Assessment Criteria

Following the recommendations by Kitchenham for evidence-based software engineering [**kitchenham˙procedures˙2004**, **kitchenham˙systematic˙2009**], each primary study must be assessed for quality, the assessment criteria must be explicitly described, applied consistently, and the overall quality of each study should be reported (e.g., high, medium, or low quality). The four Kitchenham categories used in this review are reporting quality, methodological rigor, reproducibility, and result validity. The criteria are listed below.

The following Kitchenham-style quality criteria were applied to all studies that passed the full-text screening. Each criterion was evaluated as **Yes = 1** or **No = 0**. The total score was then used to classify studies as high, medium, or low quality.

Reporting Quality

- The study states its research aim clearly.
- The model or method architecture is described in enough detail.

- The SVG/vector representation (paths, strokes, Béziers, etc.) is explained.

Methodological Rigor

- The method is appropriate for SVG/vector generation.
- The evaluation procedure is described clearly.

Reproducibility

- The study provides open-source code or enough detail to reproduce results.

Result Validity

- Quantitative metrics or qualitative SVG examples are provided.

Scoring and Classification Each criterion was scored as either **Yes = 1** or **No = 0**. The total score determined the study's quality classification:

- **6–7 points:** High quality
- **4–5 points:** Medium quality
- **0–3 points:** Low quality (may be excluded)

Kitchenham recommends that "studies should be classified as high, medium or low quality based on quality criteria" [**kitchenham˙procedures˙2004**].

Data Extraction Plan According to Kitchenham's guidelines for evidence-based software engineering, the purpose of data extraction is to systematically collect all information required to answer the research questions in a consistent and reproducible way. Kitchenham emphasises that a predefined data extraction form must be created *during the planning phase* to ensure uniformity and reduce subjective variation among reviewers [**kitchenham˙procedures˙2004**].

Predefined Data Extraction Form Kitchenham states that researchers should develop a structured data extraction form before conducting the review. This prevents ad-hoc decisions and ensures that all necessary information is collected consistently across all studies [**kitchenham˙procedures˙2004**].

Alignment with Research Questions Kitchenham notes that extracted data must be driven directly by the review's research questions. The form should therefore include only fields that support answering the research questions or performing the planned synthesis [**kitchenham˙procedures˙2004**].

Information to Extract The guidelines specify that extraction should capture both bibliographic information and structured study attributes, including study context, aims, methodology, data used, experimental setup, and results. Kitchenham emphasises that the level of detail must be sufficient for later comparison and synthesis [**kitchenham˙procedures˙2004**

Reviewer Consistency Kitchenham recommends that extracted data should be validated—ideally by a second reviewer—to minimise bias and ensure consistency. If this is not possible, reviewers should re-check their own extraction for consistency [**kitchenham˙procedures˙2004**].

Structured (Not Free-Form) Fields Kitchenham advises the use of structured or semi-structured fields to minimise subjectivity and ensure that comparable information is extracted across studies. Examples include fixed fields for model architecture, datasets, evaluation metrics, and SVG output structure [**kitchenham˙procedures˙2004**].

Avoiding Unnecessary Information Only data needed for answering the research questions or for synthesis should be extracted. Kitchenham warns that recording irrelevant details increases effort and contributes no value to the final review [**kitchenham˙systematic˙2009**].

Ensuring Comparability Across Studies Finally, Kitchenham highlights that the extraction process must ensure that the information gathered from different studies is comparable and suitable for aggregation or narrative synthesis [**kitchenham˙systematic˙2009**]. Using a consistent extraction form enables systematic comparison of SVG generation models.

2.2 Phase 2 — Conducting the Review

2.2.1 Execution of the Search Strategy

The search strings defined in Phase 1 were executed across the selected databases (Google Scholar, arXiv, and ACM Digital Library) to identify relevant literature on SVG generation and vector-based deep learning methods. Searches were performed using the conceptual string groups defined in the search strategy, with database-specific syntax adjustments where necessary. All records retrieved by these searches were exported and documented before any filtering or removal of duplicates. Table 2.1 summarises the number of studies returned by each search group and database.

Table 2.1: Search Log Summary Across Databases and Conceptual Groups

Search Group	**Google Scholar**	**arXiv**	**ACM DL**	**Total**
SVG Generation	14	12	4	30
Raster-to-Vector	7	10	2	19
Text-to-SVG	7	6	5	18
Surveys/Benchmarks/Reviews	6	3	1	10
SVG Fundamentals / Background	3	3	2	8
Total Initial findings	**37**	**34**	**14**	**85**

2.2.2 Removing of Duplications

After merging all search results, duplicate records—including repeated titles and preprint–conference duplicates—were removed. This resulted in 72 unique studies.

2.2.3 Study Selection

- Stage 1: Title, Keywords and Abstract screening.

  After removing duplicates, **72** unique studies remained. A brief title and abstract screening excluded works clearly unrelated to SVG generation (e.g., sketch-only models, raster methods, classical vectorization, or closed-source LLMs), resulting in **37** papers proceeding to full-text review.

- Stage 2: Full-text screening

A full-text assessment was conducted on the **37** studies that passed the title and abstract screening. Each paper was examined for reproducibility, architectural transparency, and whether it genuinely produced SVG or structured vector outputs as defined by the inclusion criteria. In total, **10** studies were excluded at this stage: four lacked any open-source implementation, two were benchmark papers evaluating proprietary LLMs such as GPT, Gemini, or Claude, two used non-reproducible LLM-based pipelines without sufficient methodological detail, and one relied on a purely rule-based vectorization approach. Following this eligibility check, **27** studies remained for quality assessment.

Table 2.2: Reasons for Exclusion During Full-Text Screening

Exclusion Reason	**Count**
No open-source implementation	4
Proprietary LLM benchmark studies (GPT, Gemini, Claude)	2
Non-reproducible LLM-based pipeline	2
Purely rule-based vectorization method	2
Total Excluded	10

Table 2.3: Study Selection Summary Across Screening Stages

Selection Stage	**Number of Studies**
Initial search results (all databases)	8 5
After duplicate removal	72
After title and abstract screening	37
After full-text screening	27

3 Background

Generating vector graphics computationally is harder than it seems. Unlike raster images, which can be produced by predicting pixel values, SVGs have to be structurally correct and the resulting file needs to be something a designer can actually open and work with. Otherwise, the output is not really something that can be used in practice. Getting there requires understanding the building blocks that most SVG generation models have in common, despite the variety of approaches and architectures that exist. This chapter covers those building blocks. It starts with SVG itself: what is representation of format and why its structural properties create constraints that raster generation does not have. After that, we will explain about four internal representation types that used across the analyzed models, followed by differentiable rendering, which is the technical bridge between SVG and raster image. And final subchapter closes with the supervision signals, reconstruction losses, CLIP, SDS, VSD that drive the generation process in different ways.

3.1 Vector Graphics Fundamentals

Scalable Vector Graphics (SVG) is an XML-based standard for representing two-dimensional graphics as structured text [1]. Difference between raster formats and SVG is that an SVG file is a readable document that describes how an image should be drawn rather than storing pixel values. At its core, an SVG file consists of instructions or commands that define geometric primitives such as paths, shapes and curves, styling, and hierarchical relationships between elements [2].

SVG supports a range of graphic elements, including basic shapes such as lines, rectangles, and circles, as well as text and raster image. The most flexible element is the path, which allows complex shapes to be defined using sequences of commands: moving a virtual pen, drawing straight lines, and creating smooth curves [1].

What makes SVG practically useful is editability, and that editability comes from groups and layers. Multiple shapes can be combined into groups that behave as a single unit for styling and editing [2]. Elements are rendered in a defined order, and later elements appear on top of earlier ones, making layering an intrinsic part of the representation.

This hierarchical structure is what makes SVG files editable in practice: a designer can select a group, move a layer, or change one element without affecting the rest.

Figure 3.1 shows a minimal SVG file and its rendered output. The file is plain text: each element is a single readable line that defines geometry and color directly. This is what makes SVG editable. a designer can open the file, find the element they want, and change a number.

```
<svg xmlns="http://www.w3.org/2000/svg"
     width="200" height="200">
  <!-- Face -->
  <circle cx="100" cy="100" r="80"
          fill="#FFD700"/>
  <!-- Left eye -->
  <circle cx="70" cy="80" r="10"
          fill="#333"/>
  <!-- Right eye -->
  <circle cx="130" cy="80" r="10"
          fill="#333"/>
  <!-- Sad mouth -->
  <path d="M␣65␣135␣Q␣100␣110␣135␣135"
        stroke="#333" stroke-width="5"
        fill="none"/>
</svg>
```

Figure 3.1: A minimal SVG file (left) and its rendered output (right). Each element is one readable line: the face, eyes, and mouth are independent objects defined by plain text parameters. Any of them can be selected and changed without touching the rest.

3.1.1 Paths and Bézier Curves

Most SVG graphics are built using paths. A path is a sequence of commands, *move to*, *line to*, *curve to*, that define its shape [1]. Curved segments use Bézier curves, which are controlled by a set of control points that influence curvature without lying directly on the curve.

Curved segments in paths use Bézier curves. A Bézier curve is not defined by the points it passes through but by control points that pull the curve toward them from the outside. Moving a control point reshapes the curve smoothly without breaking it.

As shown in Figure 3.2, SVG supports two types. Quadratic Bézier curves use a single control point and are simpler but less flexible. Cubic Bézier curves use two control points

and can represent almost any smooth shape [1]. Most generation models in this thesis work with cubic curves because they are expressive enough to approximate complex geometry. The control point coordinates are the core parameters that models either predict directly or adjust during optimization.

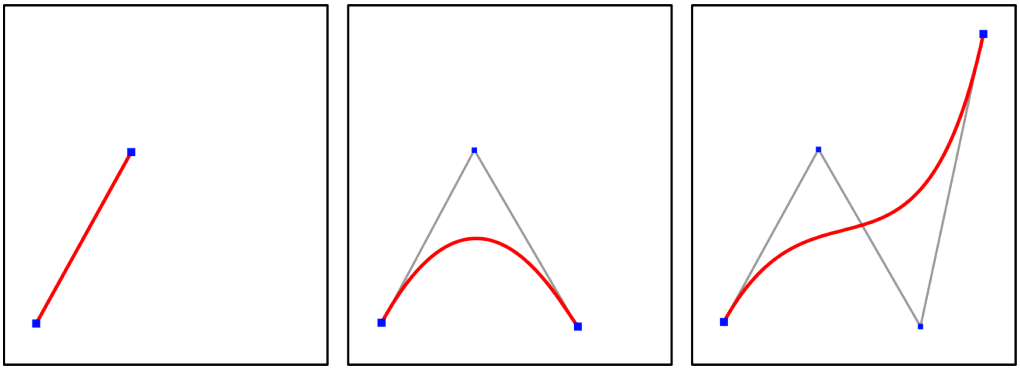

Figure 3.2: Bézier curves of grade 1, 2, and 3 (red) with their control polygons (gray). From left to right, an additional control point (blue) is added at each grade. The curve changes direction and curvature as control points are inserted or moved. Image: Wikimedia Commons [3].

3.1.2 Coordinate Systems and Transformations

SVG uses a simple coordinate system. The top-left corner of the canvas is the origin, x increases to the right, and y increases downward [2]. Every element is positioned within this grid using numeric coordinates — the same numbers you see in the SVG file.

What makes this system powerful for design is transformations. Any shape or group can be moved, scaled, rotated, or skewed by applying a transform attribute. When a transformation is applied to a group, every element inside moves together. Groups can be nested inside other groups, each with their own local coordinate frame. A logo might have a top-level group for the whole graphic, sub-groups for each component, and individual paths inside those. Moving the top-level group moves everything. Moving a sub-group moves only that part. This nested structure is what allows complex SVG files to remain organized and editable even as they grow in size.

3.2 Vector Representations

SVG generation and vectorization models do not all manipulate vector graphics the same way internally in generation processes. Obviously, the final output is always a

text-based file or SVG itself as image. What differs is the internal representation used during learning or optimization. This choice directly affects which algorithms can be applied, how stable training is, what structural properties tend to emerge and quality of final output. The following sections introduce the four representation types that appear across the models analyzed in this thesis.

3.2.1 Parametric Representation

Parametric representation is natural for SVG because SVG is already parametric. Every element in the file is defined by control point coordinates, stroke widths and colors which are readable and editable as plain text [1]. The introduction of differentiable rasterization by Li et al. [4] made it practically possible to optimize these parameters directly using image-based loss functions: a graphic is initialized as a collection of primitives, and their parameters are iteratively refined until the rendered output matches a target. Before DiffVG, this kind of gradient-based update of vector primitives was not straightforward.

3.2.2 Sequential Representation

The idea of treating vector graphics as ordered sequences of drawing commands goes back to Ha and Eck [5], who showed that recurrent networks could learn to generate stroke-based drawings by modelling them as sequences of pen movements. Vector graphics encoded this way look like a script: move the pen here, draw a line there, curve to this point. These representations share the same basic structure as a sentence, which opened the door to applying language modelling techniques directly. The downside of that approach is that whenever errors accumulate, a mistake early in a long sequence can break everything that follows.

3.2.3 Implicit Neural Representation

The use of neural networks to define geometry implicitly through learned weights was established in 3D scene representation by Mildenhall et al. [6]. In NeRF, the network acts like a function: based on given location, it tells you what occupies that space. Nothing is stored explicitly, the geometry only appears when you query the network. Thamizharasan et al. [7] adapted this idea to vector graphics by replacing explicit path coordinates with neural implicit layers that define shape boundaries indirectly. The network learns to represent the geometry as a continuous function and the resulting shapes emerge from the learned implicit representation instead of being directly parameterized.

This allows for more flexible and compact representations, but also makes it harder to control specific geometric features directly, because they are not explicitly defined as parameters.

3.2.4 Latent Representation

Latent representation works differently from the three above, it is not a vector representation in the geometric sense. There are no control points, no path commands, no shape boundaries stored anywhere. Latent space is compressing many graphics into a shared numerical embedding learned during training. Kingma and Welling [8] formalized this idea through the Variational Autoencoder: an encoder compresses an input into a short vector, and a decoder reconstructs the original from it.

For SVG generation, the actual geometry is always produced by the decoder, which translates a latent vector back into one of the other representation types: parametric paths or sequential commands. The latent space sits on top as an organizational layer. It is included here because several models use it this way and confusing it with a geometric representation leads to misunderstanding how those models actually construct their outputs.

3.3 Differentiable Rendering

Before neural networks can optimize vector graphics using image-based loss functions, there needs to be a way to connect the two spaces. Vector parameters (control points, stroke widths, colors) live in a continuous geometric space. Loss functions operate on pixel values. These two spaces are not naturally connected: traditional rasterization converts vector geometry into pixels, but this process is not differentiable, meaning gradients cannot flow back through it to update the vector parameters.

DiffVG, introduced by Li et al. [4], solves this by approximating rasterization in a way that supports gradient computation. Given a set of vector parameters as input, DiffVG renders a pixel image as output. Because the rendering step is differentiablea loss computed on the output image (whether a pixel reconstruction loss, a perceptual loss, or a signal from a pretrained semantic model) can be backpropagated all the way through to the vector parameters themselves. The system learns which control point to move, and in which direction, to reduce the error. The tricky part is handling edges and anti-aliasing — places where a tiny shift in a path boundary causes a sudden pixel change. DiffVG approximates these discontinuities in a way that still produces usable gradients [4].

Before DiffVG, optimizing vector graphics with gradient-based methods was not straightforward. Its introduction made it possible to apply the full range of image-based supervision strategies, reconstruction losses, CLIP guidance, diffusion priors directly to vector parameters.

The core idea is shown in Figure 3.3. Normally, converting vector shapes to pixels is a one-way street: you can go from shapes to pixels, but not back. DiffVG makes the middle step reversible. Once gradients can flow back through the rasterizer, any image-based loss function becomes usable for optimizing vector parameters directly.

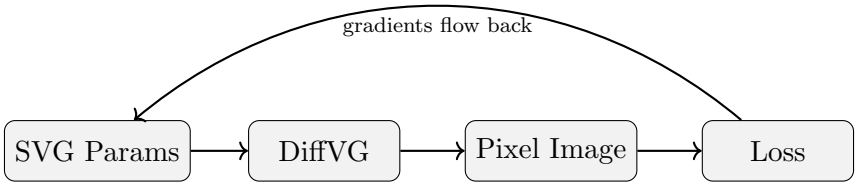

Figure 3.3: The differentiable rendering pipeline. DiffVG converts vector parameters into a pixel image. A loss compares this image to a target. Gradients then flow back through DiffVG to update the vector parameters directly [4].

3.4 Loss Functions and Semantic Guidance

Even when a model manipulates vector parameters internally, comparing outputs in pixel space is more practical than comparing vector structures directly. Most visual data exists as raster images, and pixel-based comparison gives access to well-established loss functions from computer vision. When combined with differentiable rendering, gradients from these losses flow back through the rasterizer to update vector parameters. Which loss you use matters a lot, it shapes what the model focuses on during training.

3.4.1 Reconstruction Losses

The simplest losses operate directly on pixel values. The L2 loss measures the mean squared difference between predicted and target pixels, while the L1 loss measures the mean absolute difference [9]. Both are computationally efficient and provide stable gradients when a reference image is available. The problem is that they treat every pixel the same. A pixel at a shape boundary matters more than one in a flat colored region, but L1 and L2 do not know that.

Perceptual losses address this by comparing feature representations extracted from pre-trained convolutional networks rather than raw pixel values. The formulation proposed by Johnson et al. [10] uses intermediate feature maps from VGG [11], which capture edges, textures, and object-level structure. Comparing at the feature level produces supervision signals that better reflect visual similarity as humans perceive it, and helps preserve overall structure even when exact pixel alignment differs slightly.

For tasks where shape boundary accuracy matters more than color fidelity, overlap-based losses are used. Intersection over Union (IoU) [12] measures the overlap between predicted and target regions. In vector graphics, mask-based supervision of this kind encourages correct topology and spatial consistency across layers, rather than minimizing pixel intensity differences.

All three loss types depend on having a reference raster image. In text-to-SVG settings, where no target image exists, alternative guidance strategies are needed.

3.4.2 CLIP

CLIP (Contrastive Language–Image Pretraining), introduced by Radford et al. [13], is trained on large-scale image-text pairs to learn a shared embedding space where semantically matching pairs are placed close together and mismatched pairs are pushed apart. Given an image and a text prompt, CLIP measures their similarity through separate image and text encoders, typically using cosine similarity between the resulting embeddings.

In SVG generation pipelines, the rendered vector graphic is passed through the CLIP image encoder while the text prompt is encoded separately. If the rendered output does not match the prompt semantically, the similarity score will be low. Because CLIP is differentiable with respect to its image input, and because images can be produced from vector parameters via differentiable rendering, the CLIP similarity score can serve as an optimization signal. Gradients flow from the similarity score back through the rasterizer to the SVG parameters, allowing the system to adjust shapes and colors toward better semantic alignment. CLIP does not generate images in this setup, it acts as a semantic evaluator that provides a direction for improvement.

3.4.3 Diffusion Models

Diffusion models are generative models that learn to reverse a gradual noise corruption process. During training, clean images are progressively corrupted by adding noise, and

the model learns to denoise them step by step. After training, new images can be generated by starting from random noise and iteratively denoising [14]. Stable Diffusion [15] extends this by performing the diffusion process in a compressed latent space rather than directly in pixel space, significantly reducing computational cost while preserving image quality. When conditioned on text, diffusion models learn to generate images consistent with the textual description, capturing complex visual distributions and rich semantic structure.

3.4.4 Score Distillation Sampling and VSD

Score Distillation Sampling (SDS), introduced by DreamFusion [16] for text-to-3D generation and later adopted for vector graphics by VectorFusion [17], uses a pretrained text-conditioned diffusion model as an optimization signal. The current SVG is rendered to an image, noise is added according to the diffusion schedule, and the diffusion model predicts the denoised version conditioned on the text prompt. The difference between the noisy image and the predicted denoising defines a gradient, which is backpropagated through the rasterizer to update the SVG parameters. No ground-truth image is required, the diffusion model's learned prior acts as the teacher.

A known limitation of SDS is oversaturation: because the gradient update treats the rendered image as if it were pure noise, results tend to be over-smoothed and lack fine detail. Variational Score Distillation (VSD), introduced in SVGDreamer [18], addresses this by modeling the rendered output as a particle in a distribution rather than treating it as noise directly. This produces more stable gradients and visually consistent results. Both SDS and VSD are used across several models analyzed in Chapter 6.

4 Architectural Landscape of SVG Generation

SVG generation is not a single problem with a single solution. Some methods treat vector graphics as optimization targets and adjust geometric parameters until a rendered image matches a prompt or a reference. Others learn from datasets and generate SVGs through a trained decoder. A smaller group avoids explicit geometric primitives entirely and uses neural functions to define shapes indirectly. These differences are not just technical details. They reflect fundamentally different assumptions about what a good SVG output should look like, and they lead to very different trade-offs between visual quality, structural organization, and editability.

This chapter introduces the fourteen models that form the analytical core of this thesis. They were selected from a broader research landscape that also includes sequence-based sketch models, retrieval-based vectorization systems, and more recently, large language model approaches to SVG synthesis. That wider context is sketched briefly below to clarify where the selected models sit and why they were chosen. The organizing principle is simple: these fourteen models together cover the full range of design decisions in representation and supervision, that shape how SVG structure emerges from a generation pipeline. A summary of all fourteen models is provided in Table 4.3.

Research in SVG generation spans several directions, each addressing the problem from a different starting point. An early line of work treats vector drawings as ordered sequences of commands or strokes. Inspired by language modelling, methods like SketchRNN [5] showed that neural networks can learn stroke dynamics from data. Later work extended this to full SVG command prediction, including Sketchformer [19] and related sequence models [20, 21]. These approaches established that SVG structure could be learned statistically, but long command chains make training difficult and generalization to complex scenes remains limited by the scarcity of large-scale SVG datasets.

A second direction introduced differentiable rasterization as a bridge between vector parameters and image-space supervision. DiffVG [4] made it possible to optimize vector primitives using gradients from pixel-space losses. This opened the door to reconstruction-based vectorization and, later, to semantically supervised generation using pretrained diffusion models. Most of the optimization-based models in this thesis belong to this lineage.

A third direction replaces explicit geometric primitives with neural implicit functions. Instead of optimizing Bézier coordinates directly, these methods run optimization in neural parameter space, where an MLP defines shape boundaries indirectly. This approach has connections to neural radiance fields [6] and represents an attempt to improve structural coherence through representational design rather than supervision alone.

More recently, large language models and multimodal variants have entered SVG generation. Methods such as StarVector [22], OmniSVG [23], LLM4SVG [24], and InternSVG [25] treat SVG as structured code and generate it through transformer architectures pretrained on collection of large-scale texts and images. In one sense, SVG is a natural fit for language models: the file format is sequential XML commands, not unlike a programming language. Visual alignment improves substantially compared to earlier sequence models, and these approaches handle a wide range of generation and editing tasks. However, they are not included in the main analysis here. The core question of this thesis is how specific architectural decisions impacts the structure of the output and that relationship is difficult to trace through billions of parameters. A more detailed justification is provided in the appendix.

4.1 Optimization-Based Methods

Optimization-based methods share a common premise: SVG primitives are not predicted by a trained model but initialized and then iteratively refined until the rendered output satisfies a supervision signal. The SVG is the variable being optimized, not the output of a forward pass. What varies across methods in this family is what that supervision signal is, and how geometry is represented during optimization.

The enabling technology across all of them is differentiable rasterization. DiffVG [4] connects vector parameter space to pixel space by making the rasterization step differentiable, as described in Chapter 3. This allows any loss defined on a rendered image to propagate gradients back to Bézier control points, stroke widths, and colors. The optimization loop is the same across all methods in this family: initialize primitives, render, compute loss, update parameters. What changes is the loss and the representation. Table 4.1 summarizes how each model in this group implements those differently.

Reconstruction-Supervised

The most direct use of this loop is pixel reconstruction. LIVE [26] optimizes vector primitives layer by layer, building up progressively finer approximations of a target raster image. The layer-wise strategy encourages hierarchical organization and tends to

produce outputs where individual paths retain interpretable roles. SAMVG [27] runs the same core loop but uses the Segment Anything Model [28] to initialize primitives at semantically meaningful regions rather than random positions. The core difference between their outputs is almost entirely a function of initialization, which suggests that how optimization begins matters as much as how it proceeds.

Im2Vec [29] sits at the edge of this group. It trains an encoder-decoder network to predict parametric vector primitives from a raster image in a single forward pass, supervised through differentiable rendering and reconstruction loss. At inference there is no iterative refinement. Im2Vec is included here rather than in the dataset-driven section because its primary contribution is a differentiable rendering-based training objective, not a learned generative model over SVG structure.

Semantic-Supervised

The optimization loop changes character when the supervision signal comes from a pretrained diffusion model rather than a target image. Score Distillation Sampling (SDS) [16], described in Chapter 3, allows gradients derived from a text-conditioned diffusion model to update vector parameters directly. The SVG is rendered, evaluated against a text prompt, and updated to move toward the learned distribution. No paired SVG training data is needed.

VectorFusion [17] was among the first to apply this to SVG, using a latent diffusion model [15] as the supervision backbone, as shown in Figure 4.1. SVGDreamer [18] builds on this by replacing standard SDS with Vectorized Particle-based Score Distillation (VPSD), which treats the rendered output as a particle in a learned distribution rather than a noisy sample, producing more stable gradients and reducing oversaturation, as illustrated in Figure 4.2. It also introduces explicit control over primitive count and type, organized into semantic layers. SVGDreamer++ [30] extends this with hierarchical refinement mechanisms that adapt vector organization during generation.

Neural Implicit Representation

A third variant keeps the optimization loop but changes the representation. NiVeL [7] and NeuralSVG [31] define geometry through the weights of an MLP rather than through explicit Bézier coordinates. Optimization still occurs at inference time, but the parameter space is neural rather than geometric. NiVeL decomposes the scene into layered implicit components, each defined by a neural field whose zero-level set traces a shape boundary. NeuralSVG encodes scene structure through an instance-based representation, where each semantic region is a separately optimized neural component. Because

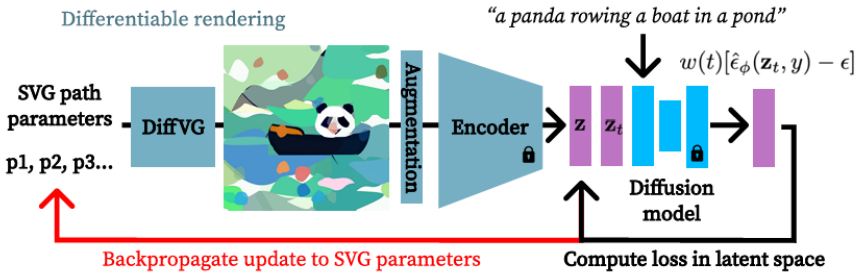

Figure 4.1: SDS optimization loop in VectorFusion [17].

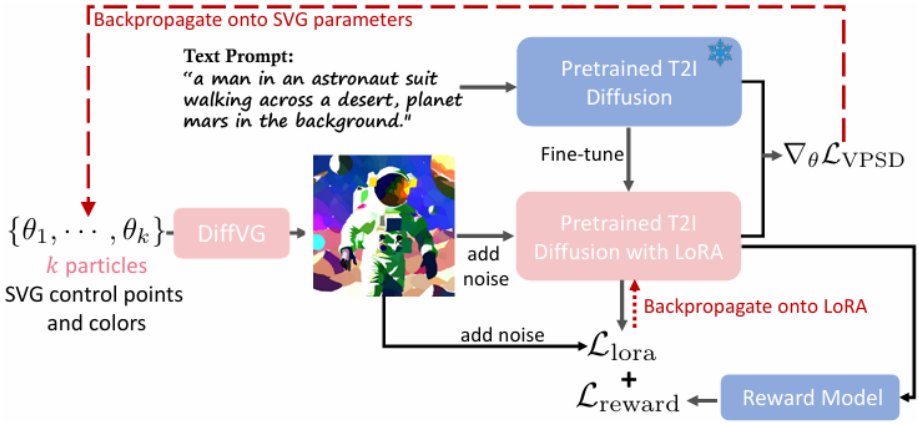

Figure 4.2: VPSD optimization loop in SVGDreamer [18].

geometry is produced through a shared function rather than a collection of independent paths, both models tend toward more globally coherent outputs, though at the cost of increased inference time.

Table 4.1 summarizes the architectural components of the optimization-based methods. For all models in this group, the "Trainables" column refers to per-instance parameters optimized at inference time rather than learned model weights. None of these methods require a training dataset; supervision is provided either by the input image directly or by a frozen pretrained model.

Table 4.1: Architectural overview of optimization-based SVG generation methods. Trainables refer to per-instance parameters optimized at inference time.

Model	Trainables	Init.	Supervision	Representation
LIVE	SVG parameters	Raster-driven	Pixel recon.	Parametric
SAMVG	None	Mask-based	None	Parametric
Im2Vec	Encoder + RNN	Image encoding	Pixel recon.	Parametric paths
VectorFusion	SVG parameters	Random/Trace	SDS	Parametric
SVGDreamer	SVG param. + LoRA	Random paths	VPSD	Parametric
SVGDreamer++	SVG param. + LoRA	Random + HIVE	VPSD + HIVE	Structured param.
NeuralSVG	MLP weights + LoRA	Saliency-based	SDS	Neural implicit
NiVeL	MLP weights	Random weights	SDS	Neural implicit

4.2 Dataset-Driven Methods

Where optimization-based methods construct each SVG from scratch at inference time, dataset-driven methods learn structural patterns from collections of vector graphics and generate through a trained forward pass. The optimization happens during training, not at inference. This changes the fundamental character of generation: outputs reflect statistical regularities in the training data rather than the iterative refinement of any single instance.

DeepSVG [32] is the foundational model in this family. It learns a hierarchical latent space over SVG path commands using a transformer-based encoder-decoder trained on the SVG-Icons dataset. The latent space separates path-level and command-level representations, allowing the model to capture both coarse shape organization and fine geometric detail. At inference, SVG commands are generated autoregressively from a sampled latent vector. DeepSVG demonstrated that structured SVG generation from a learned latent space is feasible, and also exposed the difficulty of maintaining geometric validity across long command sequences.

DeepIcon [33] applies a similar framework to icon generation specifically, trained on a curated dataset of simple icons. The domain-specific training produces outputs with more consistent style and geometry than general-purpose models, though generalization beyond the training distribution is limited by design.

LayerTracer [34] approaches the problem differently. Rather than predicting SVG commands from a latent vector, it models the construction process of vector graphics layer by layer. A diffusion-based architecture generates intermediate structural representations that guide each new layer, approximating the sequential way designers build graphics in practice. This process-aware formulation produces outputs with more meaningful layer organization than direct command prediction, with direct consequences for editability.

T2V-NPR [35] combines dataset training with a learned neural path representation. Trained on natural image and stylization data, it generates vector paths through a latent path space rather than explicit Bézier coordinates. It uses diffusion-based semantic supervision during training, but at inference it operates as a forward generative model. Its placement here reflects the fact that its structural behavior, consistent paths, stable geometry, dataset-bounded style is closer to the learned models in this section than to the per-instance optimization methods above.

SVGFusion [36] is the most architecturally complex model in this family. It first learns a continuous latent space of vector graphics using a Vector Path Variational Autoencoder (VP-VAE), trained to compress path sequences into dense latent representations. Generation then occurs through a Vector Semantic Diffusion Transformer (VS-DiT), which operates entirely in this learned latent space. This makes SVGFusion distinct from the semantic-supervised optimization methods in the previous section: where VectorFusion and SVGDreamer use diffusion gradients to guide per-instance optimization in pixel space, SVGFusion uses a diffusion process over a learned vector distribution to sample new SVGs directly. The outputs reflect the statistical structure of the training corpus rather than the convergence of any optimization loop. Table 4.2 summarizes the architectural components of the dataset-driven methods.

Table 4.2: Architectural overview of dataset-driven SVG generation methods. Training data refers to the dataset used for offline model training.

Model	**Trainables**	**Data**	**Supervision**	**Representation**
DeepSVG	VAE + Transformer	SVGIcons8	Recon. + KL	Sequential tokens
DeepIcon	Transformer	SVGIcons8	Reconstruction	Sequential tokens
SVGFusion	VP-VAE + VS-DiT	SVGX	Recon. + Denoising	Continuous latent
T2V-NPR	NPR-VAE + LoRA	FIGR-8	VSD	Neural path repr.
LayerTracer	DiT + LoRA	LayerSVG	Denoising	Layered sequences

Unlike the optimization-based group, every model here has a named training corpus. The representation column also shifts away from parametric paths toward sequential tokens, latent spaces, and layered sequences, reflecting a fundamentally different approach to how SVG structure is encoded.

Table 4.3 summarizes the fourteen models analyzed in this thesis. The *Inference* column distinguishes between methods that optimize vector parameters at inference time (*Iterative Opt.*) and methods that generate SVGs through a single trained forward pass (*Forward Dec.*). The *Supervision* column indicates the primary signal driving generation: *Opt.-based* refers to direct pixel reconstruction without a pretrained model, *Self-Sup.* to semantic signals derived from frozen pretrained models (diffusion or CLIP), and *Supervised* to training against ground-truth SVG data. Task types distinguish between *Vectorization* (raster image as input, SVG as output), *Text-to-SVG* (text prompt as

input), and *Unconditional Generation* (no input at inference — SVG sampled directly from a learned latent space).

Table 4.3: Overview of the fourteen models analyzed in this thesis, organized by generation strategy.

Model	**Task**	**Inference**	**Input**	**Output**	**Supervision**
LIVE	Vectorization	Iterative Opt.	Image	Paths	Optimization
SAMVG	Vectorization	Iterative Opt.	Image	Paths	Optimization
Im2Vec	Vectorization	Forward Dec.	Image	Paths	Self-Sup.
VectorFusion	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup.
SVGDreamer	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup.
SVGDreamer++	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup.
NiVeL	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup.
NeuralSVG	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup.
DeepSVG	Unconditional	Forward Dec.	None	Paths	Supervised
DeepIcon	Vectorization	Forward Dec.	Image	Layers/Paths	Supervised
SVGFusion	Text-to-SVG	Forward Dec.	Text	Layers/Paths	Supervised
T2V-NPR	Text-to-SVG	Forward Dec.	Text	Layers/Paths	Supervised
LayerTracer	Text-to-SVG	Hybrid	Text/Img	Layers/Paths	Supervised

The split between Iterative Opt. and Forward Dec. in the Inference column maps cleanly onto the two families: every optimization-based model refines at inference time, while every dataset-driven model generates through a single forward pass. LayerTracer is the only exception, labeled Hybrid because it combines a trained diffusion model with a per-instance vectorization step.

The fourteen models described in this chapter will be examined through two analytical lenses in the chapters that follow. The first is representation: how each model encodes vector geometry internally, and what that encoding makes possible or forecloses in terms of structural organization and editability. The second is supervision: what signal drives the generation process, and how that signal shapes the geometry that emerges. These two dimensions do not operate independently, since a model's representation constrains what supervision signals are meaningful, and the choice of supervision often determines which representations are practical. Understanding how they interact is the analytical task of the remainder of this thesis.

5 Vector Representation in SVG Generation

SVG generation models differ not only in their architectures and training objectives but also in how they represent vector graphics internally during the generation process. While all approaches ultimately produce SVG files as output, the internal representation used during learning or optimization varies considerably across methods. This variation has direct consequences for how geometry is parameterized, how supervision signals are applied, and what structural properties the resulting graphics have.

In the context of this thesis, representation refers to the intermediate form in which a model encodes vector geometry, not the final output file but the internal structure through which shapes are created, manipulated, and decoded. A vector graphic can be represented in several fundamentally different ways. It can be described as a collection of explicit geometric primitives with directly optimizable parameters. It can be encoded as an ordered sequence of drawing commands that procedurally construct the image. It can be defined implicitly through the weights of a neural network that maps indices or coordinates to shape properties. Or it can be embedded into a compact latent space from which the model samples and decodes structured output. Each of these formulations encodes the same underlying geometry differently, and this difference shapes how the model learns, how gradients propagate, and what the generated SVG looks like structurally.

Representation is therefore not a neutral implementation detail. It determines whether a model can produce clean hierarchical structure, how easily the output can be edited after generation, and how well the method scales to complex visual content. These properties are central to the comparative analysis in this thesis.

This chapter examines four primary representation families identified across the selected models: parametric, sequential, implicit neural, and latent. For each family, the analysis describes how it works technically, where it originates, and what structural tendencies it introduces in SVG generation.

5.1 Parametric Representations

Parametric representations describe vector graphics as collections of explicit geometric primitives, specifically paths defined by Bézier control points, stroke attributes, and fill colors. These parameters can be optimized directly using gradient-based methods or predicted by a neural network. The representation closely matches the SVG format itself, making the internal state of the model directly interpretable in geometric terms.

The use of parametric representations in neural SVG generation became practical with the introduction of DiffVG [4], which made the rasterization of Bézier primitives differentiable. Before this, gradients could not flow from pixel-space losses back to vector parameters. DiffVG enabled models to optimize control points and colors directly against image-based or semantic objectives, which is the foundation on which LIVE [26], SAMVG [27], VectorFusion [17], and SVGDreamer [18] are all built.

In practice, these models initialize a fixed set of vector primitives and iteratively refine their parameters so that the rendered image aligns with a target signal. The initialization strategy varies: LIVE uses a layer-by-layer approach that progressively adds paths, while SVGDreamer's SIVE module uses diffusion model attention maps to determine where primitives should be placed before optimization begins [18]. Despite these differences in initialization, all of these methods share the same underlying representational commitment, namely that shapes are stored as explicit geometric parameters that are modified directly during optimization.

The main limitation of parametric representations is that complexity is handled by increasing the number of primitives. When a model needs to capture fine visual detail, it tends to introduce many overlapping or redundant paths rather than increasing the expressiveness of individual ones. LIVE, for example, uses 160 paths across five layers to represent a single image [26]. At this scale, the output is theoretically accessible for editing since each path is a geometric object, but in practice the number and entanglement of paths makes meaningful manual editing very difficult. SVGDreamer++ explicitly identifies path entanglement as a failure mode of optimization-based parametric methods [18].

5.2 Sequential Representations

Sequential representations treat vector graphics as ordered sequences of drawing commands. Rather than storing shapes as independent geometric objects, the model generates a series of commands that describe how the graphic is constructed step by step. This mirrors the structure of SVG path data directly.

The use of sequences to represent drawings originates in work on sketch generation. Ha and Eck's SketchRNN [5] demonstrated that recurrent neural networks could learn to generate coherent hand-drawn sketches by modeling pen stroke sequences autoregressively. This established the idea that a drawing could be treated as a variable-length sequence and modeled with the same architectures used for language. DeepSVG [32] extended this approach to structured SVG icons by combining a hierarchical VAE with a transformer decoder that generates sequences of SVG path commands. DeepIcon [33] follows the same sequential formulation, learning to generate icon SVGs from command-level supervision.

Sequential representations allow the use of powerful sequence modeling architectures and can capture drawing patterns present in large datasets. However, they have known scaling limitations. Long command sequences are difficult to model reliably, and prediction errors early in the sequence propagate through the entire graphic. More fundamentally, a procedural command sequence does not encode explicit spatial hierarchy, meaning the model generates commands in order but nothing in the representation directly encodes which commands belong to which semantic region. Editability is therefore limited even when visual reconstruction quality is high.

5.3 Implicit Neural Representations

Implicit neural representations encode vector graphics through the weights of a neural network rather than through explicit geometric parameters. Instead of storing control points or commands, the model learns a function that maps indices or coordinates to shape properties such as geometry, color, and layer assignment. Shapes are defined implicitly by evaluating this function.

This approach is directly inspired by Neural Radiance Fields (NeRF) [6], which demonstrated that a compact MLP could represent a full 3D scene implicitly by mapping spatial coordinates to color and density values. NeuralSVG explicitly acknowledges this connection: each SVG is represented as a set of indices where each index corresponds to a single shape, and a small MLP learns to map these indices to Bézier control points and fill colors [31]. The network weights are the representation, not the shapes themselves. This also allows NeuralSVG to adopt an ordered representation strategy, where shapes are assigned to fixed indices during training to ensure stable correspondence between the generated output and the supervision signal.

NIVeL follows a related paradigm, representing layered vector graphics through neural functions optimized against image-based supervision. Rather than predicting individual path parameters as separate outputs, NIVeL encodes the entire layered structure implicitly, where each layer is defined by a continuous neural function over the image

domain [7]. This enables the model to produce clean layered decompositions without requiring explicit path initialization.

Implicit representations offer strong expressive power and can produce outputs with coherent global structure because the entire graphic is encoded in a single set of network weights. The main limitation is interpretability. Individual shapes are not stored as separate accessible objects, and converting the implicit output into clean, directly editable SVG primitives requires an additional extraction step. The quality of this conversion determines how usable the output is in practice.

5.4 Latent Representations

Latent representations introduce an abstraction layer by encoding vector graphics as compact embeddings in a learned latent space. Rather than representing shapes directly, the model learns a space in which structured SVG content can be sampled, interpolated, or denoised. Generation then occurs by decoding these embeddings into explicit vector primitives or command sequences.

The foundational mechanism is the variational autoencoder (VAE) [8], which trains an encoder to compress structured data into a continuous latent space and a decoder to reconstruct it. DeepSVG [32] applies this to SVG generation by encoding path command sequences into latent vectors and training a transformer decoder to reconstruct them. The latent space captures structural regularities across the training dataset, enabling the model to sample new SVGs by decoding points sampled from this space.

Latent diffusion extends this idea by applying diffusion-based generative modeling directly in the latent space of a pretrained vector encoder [15]. SVGFusion operates in this way, treating the latent vector representation as the space in which the diffusion process runs. T2V with Neural Path Representation takes a different approach, arguing that global SVG-level latent spaces are too constrained to capture diverse path combinations, and instead learns a neural path representation at the individual path level [35].

Latent representations improve scalability and generative diversity compared to direct parametric or sequential approaches. The trade-off is that edits in latent space affect the decoded output in ways that are not always predictable at the level of individual shapes. Fine-grained geometric control is harder to achieve than with parametric representations because the relationship between latent dimensions and specific visual properties is not explicit.

5.5 Hybrid Representations

In practice, many SVG generation models combine elements from more than one representation family. These combinations reflect the fact that no single representation fully addresses the competing requirements of structural correctness, editability, scalability, and visual realism.

Optimization-based diffusion methods such as VectorFusion and SVGDreamer use parametric primitives as the direct optimization target while relying on a latent diffusion model as the semantic prior. The parametric layer provides geometric interpretability, while the latent model provides semantic coherence that pure geometric optimization cannot achieve alone. SVGFusion takes a similar combined approach, encoding path-level structure into latent embeddings before applying diffusion, effectively combining latent and sequential elements in a single pipeline.

LayerTracer [34] represents a different kind of hybrid strategy. Rather than combining representation families at inference time, it integrates them during training. The model is trained on the Serpentine dataset, which contains step-by-step human SVG construction sequences, meaning the training data encodes a sequential procedural representation. However, the output the model learns to produce is a layered parametric SVG, where each layer corresponds to a meaningful visual component. The result is that editability becomes a learned property rather than an emergent one: because the model has been trained on human drawing sequences, it learns to produce outputs that are structured the way a human designer would structure them, with clean layer separation and manageable path counts.

NeuralSVG also combines ideas across families. Its core representation is implicit, with an MLP mapping shape indices to geometric parameters, but it incorporates an ordered assignment mechanism originally introduced in DeepSVG to ensure that shape indices correspond consistently to visual regions acrosstraining [31]. This ordered structure imposes a form of explicit hierarchy on what is otherwise a purely neural representation.

These hybrid strategies highlight that representation design in SVG generation is less about choosing one family and more about deciding which trade-offs to prioritize. The following chapters examine how these trade-offs manifest in the supervision strategies and qualitative outputs of the selected models.

5.6 Representation and Editability

Editability is one of the defining advantages of vector graphics and a central requirement in real design workflows. Unlike raster images, SVG graphics are expected to support direct manipulation of individual elements, including moving shapes, adjusting colors, reorganizing layers, or modifying geometry without affecting unrelated parts of the graphic. Representation plays a central role in determining how editable a generated SVG is.

Parametric representations offer the most direct path to editability because each element corresponds to explicit geometric parameters. However, this advantage depends heavily on path count and geometric organization. When models such as LIVE or SAMVG introduce large numbers of paths to match visual detail, the output becomes theoretically editable but practically difficult to work with. Sequential representations such as those used in DeepSVG and DeepIcon produce procedurally structured output, but editing a single visual element requires locating its corresponding command segment, which is not always straightforward. Implicit representations shift geometry into neural parameter space, meaning individual shapes are not directly accessible until the network output is converted into discrete primitives. How clean this conversion is determines whether the final SVG supports practical editing.

An important observation is that editability is not determined by representation type alone. Initialization strategy, supervision signals, and structural priors all contribute. Methods that incorporate segmentation, attention-based masking, or layer-aware initialization tend to produce outputs with clearer object boundaries regardless of their underlying representation. LayerTracer is a clear example: by training on human construction sequences, it learns to produce layer-separated outputs as a direct consequence of its training data rather than through explicit structural constraints.

This suggests that editability emerges from the interaction between representation, optimization strategy, and structural design rather than from representation alone. Representations that maintain explicit primitives support local editing more naturally, while representations that emphasize abstraction improve generative flexibility at the cost of direct control. Hybrid approaches attempt to balance this by preserving layered structure while allowing flexible optimization or semantic guidance.

5.7 Structural Properties Emerging from Representation

Beyond editability, representation choices directly influence the structural properties of generated SVG graphics. Structural quality refers to how well a vector graphic preserves

meaningful organization, including hierarchy, grouping, layering, and geometric consistency. While visual similarity can be achieved through many small primitives, structural properties determine whether the SVG remains interpretable and aligned with design practices.

Parametric optimization-based approaches frequently illustrate the tension between visual accuracy and structural quality. Because these methods refine primitives to minimize image-space loss, they tend to introduce redundant paths or fragmented geometry to capture fine visual detail. This improves reconstruction quality but can result in overlapping shapes and inconsistent grouping. Sequential representations produce structure implicitly through command order rather than explicit hierarchy, which can leave structural relationships ambiguous even when command prediction is accurate. Implicit neural representations allow geometry to emerge from continuous neural functions, enabling smoother global organization and layered outputs, but structural clarity depends on how carefully the conversion into discrete SVG elements is handled. Latent representations capture structural regularities from training data, which can lead to more coherent outputs compared to purely optimization-based methods, but clean hierarchy is not guaranteed unless structural signals are explicitly modeled during training.

A key observation is that structural properties in most models emerge indirectly. The majority of approaches optimize visual objectives, with structure appearing as a byproduct rather than a primary optimization target. This explains why many methods achieve strong visual fidelity but still produce SVGs with redundant primitives or weak hierarchy. Initialization strategies partially address this: methods that begin from segmentation masks, attention priors, or layer-aware starting points tend to produce cleaner organization because they reduce the need for excessive path refinement. Representation defines the space in which structural organization can appear, but initialization and supervision determine whether that structure remains coherent in the final output.

5.8 Summary

This chapter examined representation as a central analytical dimension for understanding SVG generation methods. Although all models ultimately produce SVG files, the way vector graphics are encoded during generation introduces fundamental differences in structure, editability, and visual behavior. Representation therefore acts as a unifying perspective for interpreting the design decisions underlying different approaches.

Four primary representation families were identified: parametric, sequential, implicit neural, and latent. Parametric representations provide direct geometric control and strong interpretability but may produce fragmented structures under complex optimization. Sequential representations capture procedural drawing behavior but struggle with

scalability and explicit hierarchy. Implicit neural representations enable flexible topology and expressive layered outputs while shifting optimization into neural parameter space. Latent representations offer scalable generative modeling and global style control at the cost of fine-grained geometric control.

Across these families, representations that maintain explicit primitives tend to support local editing and structural transparency, whereas representations that emphasize abstraction improve generative flexibility. Hybrid approaches combine explicit structure with learned or semantic guidance to navigate this trade-off. Editability and structural quality rarely emerge from representation alone but depend on the interaction between representation, initialization strategy, and supervision signals.

These observations provide the foundation for the next analytical dimension of the thesis. While representation defines how vector graphics are encoded, supervision determines how models learn to shape that representation. The following chapter examines supervision signals and guidance strategies, analyzing how they interact with representation choices to influence the quality, structure, and behavior of generated SVG graphics.

6 Supervision and Guidance Strategies for SVG Generation

While representation defines how vector graphics are encoded during generation, supervision determines how those representations are shaped. In SVG generation, supervision refers to the signals that guide learning or optimization so that produced vector graphics align with a target objective. These signals may come from paired datasets, reconstruction losses, semantic guidance models, or structural priors introduced during initialization.

An important distinction is that supervision in this domain extends beyond dataset labels. Many methods do not rely on ground-truth SVG targets at all. Instead, they optimize vector representations using raster reconstruction, semantic similarity, or pretrained generative models as guidance. Supervision is therefore a core architectural decision: it determines which aspects of the graphic are emphasized during generation and shapes how structure emerges.

Supervision also interacts closely with representation. Because gradients propagate through the chosen representation, the same supervision signal can produce different geometric outcomes depending on how the vector graphic is encoded. Image-based losses applied to explicit primitives may introduce additional paths to match appearance, whereas the same losses applied to implicit neural representations may reshape continuous fields. This interaction highlights that supervision and representation need to be analyzed together.

A further distinction concerns when supervision is applied. Some models learn reusable generation behavior from datasets at training time. Others apply supervision directly during inference, refining a vector representation for each individual input. This difference has significant consequences for speed, structure, reproducibility, and practical usability, and is examined in detail in Section 6.4.

The following sections examine the main supervision families used across the selected models, focusing on how different guidance strategies influence structure, editability, visual quality, and practical usability.

6.1 Reconstruction-Based Supervision

Reconstruction-based supervision guides generation by comparing a rendered SVG output with a target raster image. The objective is to minimize the difference between prediction and reference using loss functions defined in image or feature space. Common choices include pixel-wise L1 or L2 losses and perceptual losses computed from pretrained convolutional networks [10]. This paradigm is particularly common in dataset-trained models and image-to-SVG approaches.

One advantage of reconstruction-based supervision is stability. Because the target is known, gradients provide direct information about how geometry should change. Training behavior is predictable, and when datasets contain clean SVG examples, models can learn typical path counts, grouping patterns, and structural conventions. The dataset itself acts as the primary supervision signal: models such as DeepSVG [32], DeepIcon [33], and Im2Vec [29] are trained to reconstruct vector graphics from data, and the reconstruction objective is what drives learning. As discussed in Chapter 4, these models do not apply supervision at inference time, once trained, generation occurs through a forward pass.

However, reconstruction-based supervision has limits. Output quality depends heavily on dataset distribution: models trained on simplified icons may generalize poorly to complex scenes, and datasets with inconsistent SVG structure can transfer those inconsistencies to generated outputs. More importantly, reconstruction losses measure visual similarity rather than structural correctness. A model can achieve low reconstruction error while producing redundant paths or weak hierarchy, particularly when pixel-level matching dominates the objective. Without explicit structural constraints, reconstruction supervision does not guarantee editability or geometric clarity.

6.2 Semantic Guidance with Vision–Language Models

Many SVG generation tasks operate without paired SVG references. In text-to-SVG settings, the only available signal may be a textual description of the desired graphic. Semantic guidance addresses this by using pretrained vision–language models as evaluators that provide feedback about whether the rendered SVG aligns with a prompt.

As described in Chapter 3, CLIP embeds images and text into a shared representation space [13]. In SVG generation pipelines, the rendered vector graphic is passed through the image encoder and compared against the encoded text prompt. Because gradients can propagate back through the differentiable rasterizer, vector parameters can be adjusted to improve semantic alignment. This enables open-ended generation without

requiring paired SVG datasets.

Semantic guidance offers flexibility across diverse prompts and captures high-level visual semantics that are difficult to encode through handcrafted losses. At the same time, it remains indirect: the signal measures alignment in embedding space rather than geometric correctness. Optimization may therefore introduce additional primitives or unstable geometry to satisfy semantic objectives, without any guarantee of clean structure or editability.

In practice, semantic guidance is rarely used in isolation. It typically acts as an intermediate step or is combined with diffusion-based supervision, as in VectorFusion [17] and early SVGDreamer configurations. This reflects a broader pattern: CLIP guidance is most useful for conceptual alignment, but stronger perceptual signals are usually needed to achieve visually convincing results.

6.3 Diffusion-Based Supervision

Diffusion-based supervision leverages pretrained diffusion models as perceptual priors rather than relying on explicit targets or embedding similarity alone. These models capture large-scale image distributions and provide gradients that encourage rendered SVGs to move toward visually plausible solutions conditioned on a text prompt.

The most common implementation is Score Distillation Sampling (SDS), introduced by DreamFusion [16]. SDS removes the need for ground-truth images by treating a frozen diffusion model as the optimization signal — it evaluates the rendered SVG and provides gradients that push vector parameters toward outputs the diffusion model considers plausible for the given prompt. Variational Score Distillation (VSD), used in SVGDreamer [18], refines this further by reducing the oversaturation artifacts that plain SDS tends to produce.

Several models in this thesis use diffusion-based supervision as their primary guidance signal, including VectorFusion, SVGDreamer [18], SVGDreamer++ [30], NeuralSVG [31], NIVeL [7], and T2V-NPR [35].

The main advantage is expressive power. Because diffusion models encode rich visual knowledge, they can guide generation toward complex compositions and stylistic diversity that reconstruction losses alone cannot achieve. However, diffusion gradients operate in pixel space and primarily emphasize perceptual realism rather than geometric simplicity. Without additional structural constraints, optimization tends to increase primitive count to satisfy visual objectives, which directly affects editability. Inference cost is also substantially higher than for forward-decoding approaches, since each generation

requires repeated rendering and evaluation cycles.

Despite these trade-offs, diffusion-based supervision has become the dominant strategy for open-ended SVG generation because it removes the dependency on paired datasets while providing strong perceptual guidance. Most recent models combine it with representation innovations or structural priors to mitigate geometric instability, which is why supervision cannot be analyzed independently from the other design choices examined in this thesis.

6.4 Dataset Supervision vs. Per-Instance Supervision

A fundamental distinction across SVG generation methods is when supervision is applied. Dataset-supervised approaches learn generation behavior from collections of SVG graphics during training. Per-instance supervision applies guidance during generation itself, refining a vector representation for each individual input or prompt.

For dataset-supervised models, the dataset is the supervision signal. Models such as DeepSVG [32], DeepIcon [33], and Im2Vec [29] are trained on reconstruction objectives against SVG data, and once trained, generation requires no further optimization. LayerTracer [34] takes a specific variant of this approach: it is trained on a synthetically generated serpentine dataset, where vector graphics are constructed in a deliberate layer order that mimics human drawing behavior [34]. The dataset structure itself encodes the supervision prior, teaching the model to generate layered, sequential outputs that reflect design conventions rather than arbitrary path orderings.

Per-instance supervision works differently. Models such as LIVE [26], SAMVG [27], VectorFusion [17], SVGDreamer [**liSVGDreamerTexttoSVG2023**], NeuralSVG [**liNeuralSVGTexttoS**
NIVeL [**zhaoNIVeLTexttoVector2023**], and T2V-NPR [**liT2V-NPRTexttoVector2023**]
apply supervision directly during generation. No reusable generation model is trained; instead, vector parameters are optimized from scratch for each input using reconstruction losses, semantic alignment, or diffusion priors. This is computationally expensive but allows structure to emerge dynamically in response to each specific prompt or image.

Some models combine both paradigms. SVGFusion [36] learns latent representations from SVG datasets during training, then applies optimization-based refinement at inference time. This hybrid approach attempts to balance the structural consistency of dataset supervision with the flexibility of per-instance optimization. The same pattern appears in T2V-NPR [35], which learns path-level representations from data before using diffusion guidance during generation.

This distinction has direct practical implications. Dataset-supervised models produce

consistent outputs and support fast inference, but their structure is constrained by what the training data contains. Per-instance supervision offers greater adaptability but introduces variability due to initialization choices and stochastic guidance, and makes systematic evaluation across methods more difficult. Hybrid approaches mitigate some of these trade-offs but increase overall system complexity.

6.5 Structural Priors and Initialization as Supervision

Beyond explicit loss functions, many SVG generation methods rely on structural priors and initialization strategies that act as additional forms of supervision. These mechanisms are not always described as supervision, but they strongly influence how structure emerges in the resulting SVG.

Initialization determines the starting point of optimization. Random initialization provides flexibility but often leads to unstable optimization and fragmented paths. Structured initialization introduces prior knowledge about object boundaries and layout, reducing the need for excessive refinement. SAMVG [27] illustrates this by using segmentation outputs to define an initial set of semantically meaningful regions before vector optimization begins. LIVE [26] and SVGDreamer++ [**liSVGDreamerTexttoSVG2023**] use layered initialization strategies that progressively refine structure across stages. In these cases, the initialization itself constrains geometry before any loss is applied.

Structural priors can also come from learned representations. SVGFusion [36] and T2V-NPR [**liT2V-NPRTexttoVector2023**] both use latent starting points shaped by prior training, so optimization begins from a representation that already encodes typical path structures rather than from scratch. This often improves convergence speed and reduces structural noise compared to random initialization.

The interaction with diffusion supervision is particularly important here. Because diffusion gradients operate in pixel space and do not enforce hierarchy, initialization plays a critical role in determining whether structure remains interpretable. When meaningful priors are absent, diffusion-driven optimization tends to increase primitive count to match visual detail. When structured initialization is present, the same supervision signal can produce more coherent geometry with fewer paths. This is one reason why models with similar supervision strategies can produce very different structural outcomes depending on how generation is initialized.

From a broader perspective, treating initialization and structural constraints as forms of supervision helps explain observations that loss functions alone cannot account for. Methods that incorporate segmentation, layering, or learned latent starting points often generate more interpretable SVG outputs despite relying on similar perceptual objectives

as less structured alternatives.

6.6 Summary

This chapter analyzed supervision as a key dimension shaping SVG generation methods. While representation determines how vector graphics are encoded, supervision determines how these representations evolve toward desired outputs.

Reconstruction-based supervision provides explicit targets and stable optimization, but its quality depends on dataset distribution and it does not guarantee structural correctness. Semantic guidance enables generation without paired data, but its indirect nature can lead to unstable geometry. Diffusion-based supervision provides strong perceptual priors that improve visual expressiveness significantly, at the cost of increased computational load and structural complexity.

The distinction between dataset supervision and per-instance supervision clarifies how methods allocate computational effort. Dataset-trained models use the training data itself as the supervision signal and favor efficiency and consistency. Per-instance optimization offers flexibility and expressive power at the cost of runtime and reproducibility. Hybrid approaches such as SVGFusion [36] and T2V-NPR [**liT2V-NPRTexttoVector2023**] combine both, reflecting an ongoing attempt to balance speed with adaptability. LayerTracer [26] represents a particularly direct example of how dataset design can encode structural priors: its serpentine training data teaches the model to produce layered outputs without requiring any explicit structural loss.

Structural priors and initialization strategies extend the definition of supervision beyond explicit loss functions. They shape the space in which optimization occurs and strongly influence geometry quality, often more than the choice of loss function itself.

Across all supervision strategies, a recurring pattern emerges: supervision tends to prioritize visual objectives, while structural properties emerge indirectly. This explains why models with strong perceptual guidance can produce visually convincing SVGs that are difficult to edit. When analyzed together with representation, supervision provides a framework for understanding why different model architectures produce distinct structural outcomes. The next chapter examines how these properties can be measured and compared through evaluation.

7 Evaluation of SVG Generation Methods

Evaluating SVG generation is harder than evaluating raster image generation. A raster image is just pixels — if it looks right, it is right. An SVG is a structured artifact that has to look right, be geometrically stable, and remain editable. These three requirements do not always point in the same direction, and the metrics most commonly used in the literature only measure the first one.

This chapter examines how the fourteen models analyzed in this thesis are actually evaluated, where those practices fall short, and what a more complete framework would need to include. The argument is not that FID and CLIP score are useless — they capture real properties — but that relying on them alone systematically misses what makes a vector graphic useful in practice.

7.1 Evaluation of Generated SVGs

The majority of SVG generation papers evaluate their outputs using metrics borrowed directly from raster image generation research. The most common are Fréchet Inception Distance (FID) [37], CLIP-based similarity scores [13], perceptual similarity measures such as LPIPS [38], pixel-level reconstruction metrics such as MSE, and human preference studies. All of these share one property: they operate on rasterized outputs. The SVG file itself — its path structure, layer organization, and geometric properties — is invisible to every one of them.

FID measures the statistical distance between distributions of generated and reference images in a feature space derived from a pretrained Inception network [39]. Lower scores indicate closer distributional similarity. In SVG generation, outputs are rasterized before computing FID, which means the metric captures rendered appearance only. FID also has documented technical sensitivities: it is unstable at sample sizes below 10,000 [37], and scores shift significantly depending on the reference distribution. When models are evaluated against datasets that overlap with their training data, scores appear artificially favorable. Several papers in the SVG literature report FID on sets of a few

hundred rendered outputs, which produces estimates that are difficult to compare across papers.

CLIP score measures semantic alignment between a rendered image and a text prompt [13]. It is particularly useful for text-to-SVG tasks where the goal is conceptual correctness rather than pixel-accurate reconstruction. CLIP captures high-level semantic relationships effectively, but like FID it operates entirely in image space. A model can achieve strong CLIP scores while producing structurally fragmented, geometrically unstable, or completely uneditable SVG output.

LPIPS is a perceptual similarity metric that computes feature-based distances using a pretrained network [38]. It correlates better with human perceptual judgments than MSE and is used in vectorization tasks where reconstruction fidelity against a reference image matters. SAMVG uses LPIPS as part of its optimization loss [27], and it appears as an evaluation metric in several vectorization papers. The limitation is the same: it measures rendered appearance, not vector structure.

MSE is the most basic pixel-level metric — mean squared error between rendered output and reference image. It is fast to compute and easy to interpret, but sensitive to minor geometric misalignments and color averaging. LIVE explicitly avoids MSE as a training loss because it leads to blurry, color-averaged outputs [26], yet several papers still report it as an evaluation metric for comparability.

Human evaluation remains the most practically meaningful signal, but also the hardest to standardize. Papers typically show curated qualitative examples or run small user studies asking participants to rate realism, prompt alignment, or preference. These assessments capture properties that numerical metrics miss, but they are subjective, expensive to run at scale, and rarely reported with enough methodological detail to allow replication.

Table 7.1 maps which of these evaluation signals each of the fourteen models actually reports. The pattern is clear: FID and CLIP dominate text-to-SVG models, MSE and LPIPS appear in vectorization models, and structural metrics are almost entirely absent across both groups. Human evaluation is present in roughly half the papers but inconsistently defined. No model evaluates layer quality as a quantitative metric.

Three observations follow from this table. First, no model reports layer quality as a quantitative metric, despite LayerTracer, NeuralSVG, and T2V-NPR all explicitly claiming layered outputs as a contribution. Second, T2V-NPR is the only model that evaluates geometric smoothness — and it does so precisely because smoothness is central to its architectural claim, not because it is a standard metric in the field. Third, DeepIcon reports no metrics at all in the sources reviewed — its evaluation is entirely qualitative. This pattern suggests that models tend to report metrics that favor their specific contributions while structural properties remain systematically unmeasured.

Table 7.1: Evaluation signals reported across the fourteen models. ✓ = reported, – = not reported.

Model	FID	CLIP	LPIPS	MSE	User	Smooth
LIVE	–	–	–	✓	✓	–
SAMVG	✓	–	✓	✓	–	–
Im2Vec	–	–	–	✓	–	–
VectorFusion	–	✓	–	–	✓	–
SVGDreamer	✓	✓	–	–	✓	–
SVGDreamer++	✓	✓	–	–	✓	–
NiVeL	–	✓	–	–	–	–
NeuralSVG	–	✓	–	–	–	–
DeepSVG	–	–	–	✓	✓	–
DeepIcon	–	–	–	–	–	–
SVGFusion	✓	✓	–	–	✓	–
T2V-NPR	✓	✓	–	–	✓	✓
LayerTracer	✓	✓	–	✓	–	–

Emerging benchmarks have begun to recognize this gap. SVGEditBench [40] and its successor SVGEditBench V2 [41] introduce structured editing tasks and evaluate outputs using raster-based, contour-based, and description-based metrics including MSE, LPIPS, and CLIP text embeddings. SVGenius [42] proposes 18 metrics across understanding, editing, and generation dimensions. However, both benchmarks are designed primarily for LLM-based SVG processing, and neither addresses the structural properties of optimization-based or dataset-trained generation methods. The evaluation gap identified here therefore persists even in the most recent specialized benchmarks.

7.2 Evaluation Gaps

The metrics in Table 7.1 share a fundamental limitation: none of them require parsing the SVG file. They measure what the output looks like after rasterization, not what it is made of. This means they cannot detect redundant paths, fragmented geometry, missing hierarchy, self-intersecting curves, or uneditable structure. For vector graphics, these are not secondary properties.

Structural redundancy. Optimization-based methods routinely produce outputs where many small overlapping paths approximate a region that a designer would represent with one or two shapes. LIVE and VectorFusion both exhibit this behavior [26, 17]. The SVG renders correctly but is structurally opaque. No metric in Table 7.1 detects this — a

model with 300 fragmented paths and a model with 12 clean shapes can produce identical FID scores if they render similarly.

Geometric quality. Curved paths in optimization outputs are sometimes jagged, self-intersecting, or discontinuous under zoom. T2V-NPR explicitly measures smoothness and its ablation study shows that removing the neural path representation drops smoothness from 0.80 to 0.53 [35]. That is a large structural difference that FID would not detect. LIVE introduces a self-intersection penalty (Xing loss) during optimization [26], but does not report intersection counts as an evaluation metric. The gap between what models optimize for geometrically and what they report is notable.

Editability. This is the most practically important gap. The industry requirement that motivated the scope of this thesis was explicit: generated SVGs must be editable by designers using standard tools. None of the metrics in Table 7.1 measure this. Editability is occasionally demonstrated by showing that a generated element can be recolored or moved, but this is not systematically evaluated across models and no paper defines criteria for what counts as editable.

7.3 Efficiency and Reproducibility

Inference speed and reproducibility are not peripheral concerns for SVG generation — they directly determine whether a method can be used in practice. Table 7.2 summarizes code availability, hardware requirements, and inference time across all fourteen models.

The efficiency gap between optimization-based and dataset-trained methods is substantial. LIVE, the simplest reconstruction-supervised method, requires between 2,600 and 10,000 seconds per image on an RTX 3090 depending on path count [26]. VectorFusion takes 10 to 20 minutes per SVG [17]. SVGDreamer and SVGDreamer++ require 7 to 14 minutes and at least 31GB of VRAM, placing them out of reach for standard consumer hardware [18, 30]. By contrast, LayerTracer generates a layered SVG in 27 to 34 seconds [34], and SVGFusion in 24 to 36 seconds [36]. The difference is not marginal — it is two to three orders of magnitude. This gap follows directly from architecture: optimization-based methods refine parameters iteratively at inference time, while dataset-trained models generate through a single forward pass.

Hardware requirements follow the same pattern but with an important exception. Dataset-trained models are faster, but several require enterprise-grade infrastructure during training: SVGFusion trains on multiple A800 GPUs, and LayerTracer on a single A100 with

80GB VRAM. Once trained, these models can run inference on less demanding hardware, but the training cost creates a barrier for independent replication.

Reproducibility is a separate and equally important concern. Four of the fourteen models have no public code release: SAMVG, VectorFusion, DeepIcon, and SVGFusion. For these models, reported metrics cannot be independently verified or recomputed on shared test sets. NiVeL's implementation is listed as pending. This fragmentation has direct consequences for the evaluation inconsistencies described in Section 7.1: when models cannot be run, cross-model comparisons rely entirely on self-reported numbers, which may reflect favorable experimental conditions rather than typical performance.

Taken together, efficiency and reproducibility reveal a tension that runs through the entire field. The methods with the strongest structural control — optimization-based approaches — are the slowest and most hardware-intensive. The fastest methods — dataset-trained forward decoders — are often the least reproducible. No current model in the analyzed set is simultaneously fast, reproducible, and structurally strong.

Table 7.2 summarizes hardware requirements, code availability, and inference time across the fourteen models. The pattern divides cleanly along family lines: optimization-based methods are slow and generally reproducible, while several dataset-trained models are fast but lack public implementations entirely.

Table 7.2: Reproducibility and efficiency overview across the fourteen models. Inference time is per-image on reported hardware.

Model	**Code**	**Hardware**	**Inference Time**
Optimization-Based			
LIVE	Public	RTX 3090	2,600–10,000 s
SAMVG	None	RTX 3090	139–2,000 s
VectorFusion	None	Not stated	10–20 min
SVGDreamer	Public	V100 ($\geq$31GB)	7–14 min
SVGDreamer++	Public	A800 ($\geq$31GB)	$\sim$7 min
NiVeL	Pending	A100	$\sim$5 min
NeuralSVG	Public	Not stated	Unknown
Im2Vec	Public	Not stated	Unknown
Dataset-Driven			
DeepSVG	Public	2$\times$1080Ti (train)	Unknown
DeepIcon	None	Not stated	Unknown
SVGFusion	None	Multi-A800	24–36 s
T2V-NPR	Public	RTX 3090	$\sim$13 min
LayerTracer	Public	A100 (80GB)	27–34 s

7.4 Toward a Multi-Dimensional Evaluation Framework

The preceding sections establish one clear finding: current evaluation practices measure appearance well and structure poorly. FID and CLIP score are useful but insufficient. Structural metrics are almost never reported. Editability is discussed qualitatively at best. What follows proposes six evaluation dimensions that together would provide a more complete picture of SVG quality.

Semantic alignment. Does the generated SVG represent the intended concept? FID and CLIP score approximate this for text-to-SVG tasks and remain the most established signals. They should be computed on sample sizes above 1,000 and against reference distributions that do not overlap with training data.

Path economy. How efficiently does the model represent visual content? A practical measure is path count relative to visually distinct regions. NeuralSVG produces semantically accurate results with substantially fewer shapes than VectorFusion or SVGDreamer for equivalent prompts [31] — a structural difference invisible to FID. Path count is directly computable from the SVG file without rasterization.

Geometric quality. Are the paths smooth and free of self-intersections? T2V-NPR directly measures curve smoothness, and its ablation study confirms that removing the neural path representation drops the smoothness score from 0.80 to 0.53 [35]. Self-intersection count is equally computable from path data. Both are SVG-native metrics that do not require rasterization and should be standard reported signals.

Layer organization. Do generated layers correspond to semantically meaningful groups? This is the hardest dimension to quantify. A proxy is the ratio of semantically grouped elements to total path count. LayerTracer and NeuralSVG make the strongest architectural claims in this dimension, but neither reports a quantitative layer quality score [**lsongLayerTracerCognitiveAlignedLayered2025**, 31]. Developing a standardized layer metric remains an open problem.

Editability. Can a designer modify the output using standard tools? This requires human evaluation — importing the SVG into Illustrator or Figma and assessing whether elements can be selected, recolored, and repositioned independently. Figure **??** shows

what target editability looks like in practice. Based on their architectural design, SVG-Fusion, LayerTracer, and NeuralSVG are the strongest candidates in the analyzed set, though this has not been formally tested across all models.

Efficiency. What hardware and time does generation require? Table 7.2 provides the data. This dimension is especially relevant for industrial deployment, where generation speed and infrastructure cost are primary constraints.

Table 7.3 applies these six dimensions to all fourteen models based on their architectural properties. Most ratings reflect design characteristics rather than measured outcomes — a direct consequence of the evaluation gap this chapter identifies. The pattern that emerges is consistent: no single model scores well across all six dimensions simultaneously. Optimization-based methods tend to score well on semantic alignment and layer organization but poorly on path economy, geometric quality, and efficiency. Dataset-trained methods score better on efficiency and path economy but vary widely on semantic alignment and editability depending on training data scope. Neural implicit methods occupy a middle ground — strong on geometry and layers, but slow.

This is not a ranking of models — it is a map of the trade-offs that current architectures navigate. The absence of a model that scores well on all six dimensions is itself a finding: it reflects that the field has optimized for what is easy to measure rather than what is useful in practice. Closing that gap is the practical motivation behind the analysis in this thesis.

Table 7.3: Proposed multi-dimensional evaluation framework applied to the fourteen models. Ratings reflect architectural design properties: + = strong, ˜ = partial, - = weak. * = empirically tested in paper.

Model	**Semantic Align.**	**Path Economy**	**Geom. Quality**	**Layers**	**Edit.**	**Effic.**
Optimization-Based						
LIVE	˜	˜	˜*	+	˜	-
SAMVG	˜	+	˜	+	+	˜
VectorFusion	+	-	-	-	-	-
SVGDreamer	+	˜	˜	+	˜	-
SVGDreamer++	+	˜	˜	+	˜	-
NiVeL	+	+	+	+	˜	-
NeuralSVG	+	+*	+	+	+	-
Im2Vec	˜	˜	˜	-	˜	+
Dataset-Driven						
DeepSVG	˜	˜	˜	-	˜	+
DeepIcon	˜	+	+	+	+	+
SVGFusion	+	+	+	+	+	+
T2V-NPR	+	+*	+*	+	+	-
LayerTracer	+	+*	˜	+	+	+

8 Comparative Synthesis

The preceding chapters established what SVG generation methods do and how they should be evaluated. This chapter examines what they reveal when compared directly. Using the representational and supervisory dimensions introduced in Chapters 5 and 6, and the evaluation framework from Chapter 7, the analysis identifies recurring patterns across the fourteen models that explain why certain approaches produce outputs that are structurally clean and editable while others prioritize visual realism at the cost of usability.

The findings are organized around four observations: how representation shapes structural outcomes, how supervision influences geometric behavior, how the realism–editability trade-off manifests concretely across models, and what conditions consistently lead to outputs that satisfy both visual and structural requirements.

8.1 Representation and Structural Outcomes

The internal representation of vector graphics is the strongest predictor of structural quality across the analyzed models. Representation determines what kinds of structures are possible before any optimization or training begins.

Table 8.1 maps each model's representation and supervision choices to the structural outcomes observed in its outputs. The patterns this table makes visible are the subject of the analysis that follows.

Parametric representations, used in optimization-based methods, operate directly on Bézier control points, colors, and stroke parameters. This gives clear geometric control and enables gradient-based updates. However, when supervision operates primarily in raster space, parametric methods tend to increase primitive count to minimize visual error. The result is structurally dense output — visually accurate but fragmented. LIVE and VectorFusion both exhibit this behavior, where each additional path reduces reconstruction loss without contributing to semantic structure [26, 17].

Sequential representations model SVG as ordered command sequences. This captures

Table 8.1: Representation type, supervision signal, and observed structural outcomes across the fourteen analyzed models.

Representation	**Supervision**	**Models**	**Structural Outcome**
Parametric	Pixel recon.	LIVE, SAMVG	High path count, accurate boundaries, fragmented structure
Parametric	Pixel recon.	Im2Vec	Compact paths, limited topological complexity
Parametric	SDS	VectorFusion	Semantically rich, dense primitives, low editability
Parametric	VPSD	SVGDreamer, SVGDreamer++	Improved diversity, object-aware decomposition, still dense
Neural implicit	SDS	NiVeL, NeuralSVG	Fixed layer count, constrained structure, moderate editability
Sequential tokens	Reconstruction	DeepSVG, DeepIcon	Clean paths within training domain, poor generalization outside it
Neural path latent	VSD	T2V-NPR	Smooth geometry, stable paths, dataset-bounded style
Continuous latent	Denoising	SVGFusion	Logical construction order, coherent layers, fast inference
Layered sequences	Denoising	LayerTracer	Designer-aligned layers, explicit construction logic, strong editability

procedural drawing logic and produces coherent structure in constrained settings — DeepSVG generates well-organized icon-level paths because the training data encodes implicit structural regularities [32]. The limitation is scalability: long command chains amplify error propagation, and global structural consistency degrades as scene complex-

ity increases.

Neural implicit representations shift structure into representation space itself. NeuralSVG and NiVeL define shapes through MLP weights rather than explicit path coordinates [31, 7]. Because all geometry passes through a shared neural function, the representation constrains how primitives can be arranged. Layering emerges architecturally rather than through optimization dynamics. This acts as a structural regularizer — fewer primitives, clearer decomposition — but the mapping between neural parameters and editable SVG primitives becomes less direct.

Latent representations, used in SVGFusion and T2V-NPR, generate through learned embeddings that encode dataset-level structural patterns [36, 35]. This allows the model to inherit consistent path organization from training data, producing more coherent structure than unconstrained optimization. The trade-off is reduced direct control over primitive composition — structure is mediated through the latent space rather than directly accessible.

The pattern across all four representation types is consistent: representations that constrain decomposition produce cleaner outputs. Representations that maximize flexibility produce more complex, harder-to-edit outputs. Structural quality is partly a representational property, not only an optimization outcome.

8.2 Supervision and Geometric Behavior

Where representation defines what structures are possible, supervision determines which of those structures are encouraged. The choice of supervision signal shapes geometric behavior directly — influencing primitive count, curve stability, and layer organization.

Reconstruction supervision, used in dataset-trained methods, encourages models to reproduce observed examples. When paired with sequential or latent representations, this produces stable geometry within the training distribution. DeepSVG and DeepIcon generate consistent path organization because reconstruction loss implicitly encodes structural regularities from the dataset [32]. Outside the distribution, geometry quality degrades.

Pixel reconstruction supervision, used in pure optimization methods, encourages accurate boundary alignment without penalizing redundant primitives. As a result, geometry fragments as additional paths are introduced to reduce visual error. LIVE's analysis of vectorization outputs confirms that pixel-driven supervision prioritizes local accuracy over global structure [26].

Diffusion-based supervision provides strong distributional priors through SDS and related techniques [16]. This enables text-to-SVG generation without paired vector data but introduces characteristic geometric effects. Because diffusion supervision operates in raster space and prioritizes realism, optimization increases primitive count to approximate high-frequency visual detail. VectorFusion and SVGDreamer achieve strong visual results through this mechanism while producing structurally dense SVGs [17, 18].

Methods that combine semantic guidance with constrained representations — NeuralSVG using SDS within a neural implicit space, T2V-NPR using VSD within a learned latent path space — exhibit more stable geometry because supervision acts within a restricted domain. Rather than freely increasing primitives, optimization is bounded by the representation, which reduces fragmentation and improves structural coherence.

The key observation is that supervision and representation do not act independently. The same diffusion supervision signal produces very different geometric outcomes depending on whether it operates over explicit Bézier paths or a constrained neural parameter space. This interaction explains why models with similar visual performance can differ substantially in structural quality.

8.3 The Realism–Editability Trade-off

The most consistent pattern across the fourteen models is the tension between visual realism and editability. This tension is not theoretical — it is observable in specific models and has direct practical consequences.

Methods driven by diffusion-based supervision achieve visually convincing outputs by refining geometry until the rendered image aligns with learned image priors. This refinement increases structural complexity in ways that undermine usability. LIVE, following the optimization protocol also adopted by VectorFusion, represents a single image using up to 160 paths distributed across multiple layers [18]. At this scale, selecting or modifying a single visual component requires identifying and adjusting dozens of overlapping primitives in standard design tools. SVGDreamer++ explicitly acknowledges this failure mode, noting that optimization-based approaches produce outputs where objects are entangled across paths, making component-level editing effectively impossible [18].

This structural cost comes with a genuine visual benefit. VectorFusion and SVGDreamer consistently produce the most visually rich and semantically detailed outputs among the analyzed methods [17, 18]. The same mechanism that fragments structure — unconstrained primitive growth guided by strong diffusion priors — is precisely what enables photorealistic rendering quality. The trade-off is genuine: neither side is simply better. The appropriate choice depends on whether the intended use prioritizes visual output

quality or downstream editability.

Models that constrain representation tend to produce outputs that are easier to edit, even when visual abstraction is introduced. NeuralSVG represents complex scenes using a small fixed number of layers, improving usability despite reduced photorealistic detail [31]. Dataset-trained models such as DeepSVG and DeepIcon inherit structural simplicity from their training data, producing directly editable outputs bounded by the style distribution of the dataset [32].

LayerTracer represents a distinct route toward editability. Rather than constraining an optimization process, it is trained directly on layered construction sequences that reflect how designers build SVG graphics step by step [34]. Editability is not an emergent property but a learned one — the model produces layered outputs because it was explicitly supervised to construct graphics that way. This makes LayerTracer structurally closer to designer workflows than any optimization-based method analyzed here.

The realism–editability trade-off emerges from the interaction between supervision strength and representational constraints. When supervision strongly prioritizes visual similarity without structural regularization, primitive counts increase and editability degrades. When representation enforces decomposition or limits complexity, editability improves but visual detail is reduced. Hybrid approaches that integrate structural priors with semantic guidance — as in NeuralSVG and T2V-NPR — suggest that both objectives can be partially reconciled, though no current model satisfies both simultaneously at the level required for professional design workflows.

8.4 Conditions for Clean and Editable SVG

Across the fourteen models, five conditions consistently correlate with cleaner and more editable SVG outputs. These are not isolated design choices but interacting factors that together determine whether a generated SVG is structurally usable.

Primitive count control. Methods that limit path count — either through explicit budgets, latent space constraints, or neural parameterization — produce outputs with less redundancy. Unconstrained primitive growth, common in pixel-supervised optimization, is the primary source of structural fragmentation.

Semantically aligned decomposition. SVGs become more editable when primitives correspond to meaningful visual components rather than arbitrary geometric fragments. Initialization strategies that use segmentation or saliency maps (SAMVG, NeuralSVG) and architectures that model layer-wise construction (LayerTracer) both encourage this alignment.

Constrained supervision domain. When supervision acts within a structured representation space rather than directly in raster space, geometric behavior is more regulated. T2V-NPR and NeuralSVG both demonstrate that diffusion-based semantic guidance produces more stable paths when the optimization domain is constrained.

Informed initialization. Random initialization with unconstrained optimization leads to unstable structure. Semantically meaningful starting points — from segmentation masks, attention maps, or learned latent embeddings — reduce the need for excessive primitive refinement and improve structural coherence from the outset.

Design-aware architectural priors. Models that incorporate layering, sequential construction, or scene composition logic produce outputs that more closely reflect designer intent. LayerTracer is the clearest example — editability is architectural rather than incidental.

Table 8.2: Conditions for clean and editable SVG and the models that satisfy each.

Condition	Models	Effect
Primitive count control	NeuralSVG, NiVeL, T2V-NPR	Reduces fragmentation
Semantic decomposition	SAMVG, NeuralSVG, LayerTracer	Improves element grouping
Constrained supervision	NeuralSVG, T2V-NPR	Stabilizes geometry
Informed initialization	SAMVG, SVGDreamer, NeuralSVG	Reduces redundant refinement
Design-aware priors	LayerTracer, SVGFusion	Editability by design

No current model satisfies all five conditions simultaneously. NeuralSVG comes closest among optimization-based methods. LayerTracer comes closest among dataset-trained methods. The gap between them reflects the broader realism–editability trade-off: NeuralSVG produces more flexible outputs but requires more compute, while LayerTracer produces consistently structured outputs bounded by its training distribution.

8.5 Implications for Future Work

The patterns identified in this chapter point toward two directions that appear most promising for advancing SVG generation toward outputs that are both visually convincing and practically usable.

The first is tighter integration between representation design and supervision. The analysis consistently shows that supervision operating within constrained representations produces better structural outcomes than supervision operating in raster space. Future models that explicitly co-design representation constraints and supervision signals — rather than treating them as independent architectural decisions — are likely to make

more progress on the realism–editability trade-off than approaches that improve either dimension alone.

The second is the formalization of editability as a training objective rather than an evaluation afterthought. LayerTracer demonstrates that models can be trained directly on designer construction logic, producing editability by design rather than by constraint. Extending this approach to richer scene types, larger datasets, and more expressive representations would address the generalization limitations that currently bound dataset-trained methods. Combining this with the geometric stability advantages of neural implicit representations represents a direction that none of the fourteen analyzed models has yet explored fully.

9 Conclusion

This thesis examined how SVG generation methods construct vector graphics and why their structural outcomes differ. The analysis was organized around two dimensions (representation and supervision) and applied across fourteen open-source models spanning optimization-based, neural implicit, and dataset-driven approaches. The following sections address each research question in turn and summarize the contributions of the work.

Answers to the Research Questions

RQ1: What are the main properties and structures that define an editable and high-quality SVG?

An editable SVG requires a small number of semantically meaningful paths, a clear layer structure where visual components correspond to distinct groupings, and stable geometry without redundant or overlapping primitives. Visual quality and editability are separable properties: a graphically rich output can simultaneously be difficult to edit if its paths are fragmented or entangled. Chapter 3 established these properties as part of the foundational definition of SVG, and Chapter 8 identified five conditions under which current models satisfy them.

RQ2: What open-source methods currently exist for generating SVGs from different input modalities?

The review identified two broad methodological families. Optimization-based methods (LIVE, SAMVG, Im2Vec, VectorFusion, SVGDreamer, SVGDreamer++, NiVeL, and NeuralSVG) operate by iteratively refining vector parameters to match a target image or semantic description. Dataset-driven methods (DeepSVG, DeepIcon, LayerTracer, T2V-NPR, and SVGFusion) learn generation from collections of existing SVGs. Chapter 4 described their input modalities, which span raster images, text prompts, and unconditional generation from latent distributions. Large language model approaches were reviewed but excluded from the main analysis, as discussed in Chapter 1.

RQ3: What types of deep learning architectures are used for SVG generation?

The analyzed models employ four distinct architecture types: parametric optimization over Bézier primitives using differentiable rasterization, transformer-based hierarchical sequence models, MLP-parameterized neural implicit representations, and latent diffusion architectures with vector-specific decoders. Chapter 4 introduced each type, and Chapter 5 analyzed how representational choices within each type determine the structural properties of the output.

RQ4: What kinds of structure do these models produce?

Structural outcomes vary substantially across the analyzed models and correlate with representation choice more than with visual quality. Parametric optimization methods tend to produce high path counts with fragmented geometry. Neural implicit and dataset-driven methods produce more constrained outputs with interpretable layer organization, at the cost of reduced visual complexity. Chapter 5 traced how representation shapes these structural outcomes, Chapter 6 examined how supervision signals interact with them, and Chapter 8 mapped the resulting trade-offs across all fourteen models.

RQ5: How can the quality and editability of generated SVG be evaluated?

Current evaluation practice relies primarily on metrics adapted from raster image synthesis (FID, CLIP similarity, LPIPS, and MSE) which measure visual fidelity but not structural properties. Chapter 7 proposed a multi-dimensional evaluation framework covering six dimensions: semantic alignment, path economy, geometric quality, layer organization, editability, and efficiency. The chapter also identified SVGEditBench and SVGenius as emerging benchmarks, while noting that both were designed for language-model-based SVG generation and do not directly apply to the optimization and dataset-trained models analyzed here.

RQ6: What is the observed quality of SVG outputs across different models?

No model in the analyzed set performs well across all six evaluation dimensions simultaneously. Optimization- based models achieve the strongest visual fidelity but produce the least editable outputs. Dataset-driven models produce structurally cleaner SVGs within constrained domains. LayerTracer and SVGFusion come closest to balancing visual quality with structural usability among the analyzed set. Chapter 8 summarized these findings in a conditions table and identified the specific architectural factors that distinguish models on the editability-realism axis.

Contributions

The primary contribution of this thesis is an analytical framework for understanding SVG generation in terms of representation, supervision, and their structural consequences. This framework provides a consistent basis for comparing methods that differ substantially in architecture and training strategy, and makes explicit a trade-off (between visual realism and editability) that is present across the field but rarely stated directly.

A secondary contribution is a multi-dimensional evaluation perspective that extends beyond pixel-level metrics to structural and practical criteria. The metrics coverage table in Chapter 7 documents what each model actually reports versus what would be needed for a complete structural assessment, and identifies the specific evaluation gaps that remain open.

The analysis also produced a set of conditions for clean and editable SVG output (primitive count control, semantically aligned decomposition, structured supervision, informed initialization, and design-logic priors) which represent a checklist for future model design that does not currently exist in the literature.

Limitations

The analysis is bounded by the set of publicly available, open-source models with documented implementations. Several models with reported strong results, including VectorFusion and DeepIcon, did not have accessible code at the time of writing, limiting the depth of implementation-level analysis for those methods. Quantitative comparison across models was not feasible because reported metrics are heterogeneous: different models evaluate on different datasets, use different implementations of shared metrics, and report results under different experimental conditions. The evaluation framework proposed in Chapter 7 is conceptual rather than empirically validated.

Future Work

Three directions follow directly from the findings.

Evaluation protocols that explicitly measure structural quality remain the most immediate gap. A benchmark combining path economy, layer coherence, and editability scores

alongside visual fidelity metrics would enable direct comparison across optimization-based and dataset-driven methods for the first time. This is tractable to build using existing SVG tools.

Hybrid architectures that combine fast feed-forward generation with constrained vector refinement represent the most promising near-term direction for closing the realism-editability gap. The conditions identified in Chapter 8 suggest that structural constraints can be introduced at the representation level without necessarily sacrificing visual quality, but this has not been demonstrated in a single model.

Supervision within structured representation spaces (path-level latent constraints, layer consistency objectives, construction sequence learning) appears to regulate the fragmentation that characterizes purely raster-space optimization. SVGDreamer's VPSD approach and LayerTracer's construction-sequence supervision both point in this direction from different starting points. Converging these two approaches is a concrete research question.

Bibliography

[1] J. David Eisenberg. *SVG Essentials: Producing Scalable Vector Graphics with XML*. Repr. Beijing Köln: O'Reilly, 20.

[2] A. Quint. "Scalable Vector Graphics". In: *IEEE Multimedia* 10.3 (July 2003), pp. 99–102.

[3] Wikimedia Commons. *Bézier curve — Wikimedia Commons*. Public domain. 2024.

[4] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, Jonathan Ragan-Kelley. "Differentiable Vector Graphics Rasterization for Editing and Learning". In: *ACM Trans. Graph.* 39.6 (Nov. 2020), 193:1–193:15.

[5] David Ha, Douglas Eck. *A Neural Representation of Sketch Drawings*. May 2017.

[6] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, Ren Ng. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. Aug. 2020.

[7] Vikas Thamizharasan, Difan Liu, Matthew Fisher, Nanxuan Zhao, Evangelos Kalogerakis, Michal Lukac. *NIVeL: Neural Implicit Vector Layers for Text-to-Vector Generation*. May 2024.

[8] Diederik P. Kingma, Max Welling. *Auto-Encoding Variational Bayes*. Dec. 2022.

[9] Ian Goodfellow, Yoshua Bengio, Aaron Courville. *Deep Learning*. MIT Press, 2016.

[10] Justin Johnson, Alexandre Alahi, Li Fei-Fei. *Perceptual Losses for Real-Time Style Transfer and Super-Resolution*. 2016.

[11] Karen Simonyan, Andrew Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *ICLR*. 2015.

[12] Mark Everingham et al. "The PASCAL Visual Object Classes (VOC) Challenge". In: *IJCV*. 2010.

[13] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. Feb. 2021.

[14] Jonathan Ho, Ajay Jain, Pieter Abbeel. "Denoising Diffusion Probabilistic Models". In: *NeurIPS*. 2020.

[15] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Björn Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2021.

[16] Ben Poole, Ajay Jain, Jonathan T. Barron, Ben Mildenhall. *DreamFusion: Text-to-3D Using 2D Diffusion*. Sept. 2022.

[17] Ajay Jain, Amber Xie, Pieter Abbeel. *VectorFusion: Text-to-SVG by Abstracting Pixel-Based Diffusion Models*. Nov. 2022.

[18] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, Qian Yu. *SVGDreamer: Text Guided SVG Generation with Diffusion Model*. Dec. 2024.

[19] Leo Sampaio Ferraz Ribeiro, Tu Bui, John Collomosse, Moacir Ponti. *Sketchformer: Transformer-based Representation for Sketched Structure*. Feb. 2020.

[20] Yaroslav Ganin, Tejas Kulkarni, Igor Babuschkin, S. M. Ali Eslami, Oriol Vinyals. *Synthesizing Programs for Images Using Reinforced Adversarial Learning*. Apr. 2018.

[21] Raphael Gontijo Lopes, David Ha, Douglas Eck, Jonathon Shlens. *A Learned Representation for Scalable Vector Graphics*. Apr. 2019.

[22] Juan A. Rodriguez, Abhay Puri, Shubham Agarwal, Issam H. Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, Marco Pedersoli. *StarVector: Generating Scalable Vector Graphics Code from Images and Text*. May 2025.

[23] Yiying Yang, Wei Cheng, Sijin Chen, Xianfang Zeng, Fukun Yin, Jiaxu Zhang, Liao Wang, Gang Yu, Xingjun Ma, Yu-Gang Jiang. *OmniSVG: A Unified Scalable Vector Graphics Generation Model*. May 2025.

[24] Ximing Xing, Juncheng Hu, Guotao Liang, Jing Zhang, Dong Xu, Qian Yu. *Empowering LLMs to Understand and Generate Complex Vector Graphics*. Mar. 2025.

[25] Haomin Wang et al. *InternSVG: Towards Unified SVG Tasks with Multimodal Large Language Models*. Oct. 2025.

[26] Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, Humphrey Shi. *Towards Layer-wise Image Vectorization*. June 2022.

[27] Haokun Zhu, Juang Ian Chong, Teng Hu, Ran Yi, Yu-Kun Lai, Paul L. Rosin. *SAMVG: A Multi-stage Image Vectorization Model with the Segment-Anything Model*. Dec. 2023.

[28] Alexander Kirillov et al. *Segment Anything*. Apr. 2023.

[29] Pradyumna Reddy, Michael Gharbi, Michal Lukac, Niloy J. Mitra. *Im2Vec: Synthesizing Vector Graphics without Vector Supervision*. Apr. 2021.

[30] Ximing Xing, Qian Yu, Chuang Wang, Haitao Zhou, Jing Zhang, Dong Xu. "SVGDreamer++: Advancing Editability and Diversity in Text-Guided SVG Generation". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47.7 (July 2025), pp. 5397–5413.

[31] Sagi Polaczek, Yuval Alaluf, Elad Richardson, Yael Vinker, Daniel Cohen-Or. *NeuralSVG: An Implicit Representation for Text-to-Vector Generation*. 2025.

[32] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, Radu Timofte. "DeepSVG: A Hierarchical Generative Network for Vector Graphics Animation". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, H. Lin. Vol. 33. Curran Associates, Inc., 2020, pp. 16351–16361.

[33] Qi Bing, Chaoyi Zhang, Weidong Cai. *DeepIcon: A Hierarchical Network for Layer-wise Icon Vectorization*. Oct. 2024.

[34] Yiren Song, Danze Chen, Mike Zheng Shou. *LayerTracer: Cognitive-Aligned Layered SVG Synthesis via Diffusion Transformer*. Aug. 2025.

[35] Peiying Zhang, Nanxuan Zhao, Jing Liao. *Text-to-Vector Generation with Neural Path Representation*. May 2024.

[36] Ximing Xing, Juncheng Hu, Jing Zhang, Dong Xu, Qian Yu. *SVGFusion: Scalable Text-to-SVG Generation via Vector Space Diffusion*. Mar. 2025.

[37] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, Sepp Hochreiter. "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[38] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, Oliver Wang. "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2018, pp. 586–595.

[39] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, Zbigniew Wojna. "Rethinking the Inception Architecture for Computer Vision". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2016, pp. 2818–2826.

[40] Kunato Nishina, Yusuke Matsui. "SVGEditBench: A Benchmark Dataset for Quantitative Assessment of LLM's SVG Editing Capabilities". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 2024, pp. 8142–8147.

[41] Kunato Nishina, Yusuke Matsui. "SVGEditBench V2: A Benchmark for Instruction-Based SVG Editing". In: *arXiv preprint arXiv:2502.19453* (2025).

[42] Siqi Chen et al. "SVGenius: Benchmarking LLMs in SVG Understanding, Editing and Generation". In: *Proceedings of the 33rd ACM International Conference on Multimedia*. 2025.

List of Figures

Appendix

Great table.

Headline 1	**Headline 2**
Titel 1	Description 1
Titel 2	Description 2

Table 1: Overview in a great table.

Table of chosen approaches

Inference Time Overview

Model	Task Type	Inference	Input	Output	Supervision	Core Idea
VectorFusion	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Distills diffusion priors via differentiable rasterization.
SVGFusion	Text-to-SVG	Forward Dec.	Text	Paths	Supervised	Continuous latent space learning using SVGX dataset.
SVGDreamer	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Semantic-driven vectorization (SIVE) and Particle SDS.
SVGDreamer++	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Hierarchical optimization with adaptive primitive control.
T2V-NPR	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Dual-branch VAE learns path latent space constraints.
LayerTracer	System/Pipeline	Hybrid	Text/Img	Layers/Paths	Supervised	Models designer construction logic via DiT sequences.
Im2Vec	Vectorization	Forward Dec.	Img	Paths	Self-Sup	Differentiable rasterizer allows learning from appearance.
SAMVG	Vectorization	Iterative Opt.	Image	Paths	Opt.-based	Uses SAM for semantic initialization in vectorization.
NIVeL	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Implicit neural layers as optimization domain for SDS.
NeuralSVG	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Scene encoded in MLP weights for layered shapes.
LIVE	Vectorization	Iterative Opt.	Image	Layers/Paths	Opt.-based	Layer-wise hierarchical optimization of vector graphs.
DeepSVG	Generation	Forward Dec.	None	Paths	Supervised	Hierarchical Transformer for shape/command disentanglement.
DeepIcon	Vectorization	Forward Dec.	Image	Layers/Paths	Supervised	Hierarchical network for direct image-to-SVG translation.

Training Architecture – Optimization-Based Methods

Model	Trainables	Training Data	Init.	Supervision	Representation	Priors
LIVE	SVG parameters	None	Raster-driven	None	Parametric (flat)	None
SAMVG	None	None	Mask-based	None	Parametric (mask-derived)	SAM
VectorFusion	SVG parameters	None	Random / Trace	SDS (Stable Diff.)	Parametric (flat)	Stable Diff.
SVGDreamer	SVG parameters / LoRA	None	Random paths	VPSD	Parametric (flat)	Stable Diff.
SVGDream++	SVG parameters / LoRA	None	Random + HIVE	VPSD + HIVE loss	Structured parametric	Stable Diff. + SAM
NeuralSVG	MLP weights + colors/ LoRA	None	Saliency-based	SDS	Implicit neural fields	Stable Diff.
NIVeL	MLP weights + layer colors	None	Random weights	SDS	Implicit neural fields	DeepFloyd + CLIP

Training Architecture – Dataset-Trained Methods

Model	Trainables	Training Data	Init.	Supervision	Representation	Priors
DeepSVG	VAE (Transformer) weights	SVGIcons8	Latent sampling	Recon. + KL	Sequential path tokens	None
DeepIcon	Transformer weights	SVGIcons8	CLIP embedding	Reconstruction	Sequential path tokens	CLIP encoder
SVGFusion	VP-VAE + VS-DiT	SVGX	Random latent	Recon. + Denoising	Continuous latent space	CLIP + DINO-v2
T2V-NPR	NPR-VAE + Latents + LoRA	FIGR-8	Random path latents	VSD	Neural Path Repr.	Stable Diff. + CLIP
Im2Vec	Encoder + RNN + 1D CNN	Raster images	Image encoding	Reconstruction	Parametric paths	None
LayerTracer	DiT + LoRA	Serpentine dataset	Diffusion noise	Denoising loss	Layered pixel sequences	Flux DiT + Florence-2

Note: - Order in table: non-learning -¿ optimization-based -¿ implicit -¿ dataset-trained -¿ systems.

- For optimization-based methods, “trainables” means to per-instance SVG parameters optimized at inference time rather than learned model weights.

- Trained on text prompts means, optimize SVG paramets at inference time using a frozen pretrained model as semantic prior.

- Semantic distillation means supervision signal got by backpropagating semantic alignments scores from Stable Diff. model to optimize target representation.

- SVGFusion, first trains VAE on SVGX dataset to learn a latent representations of SVGs, then diffusion transformer is trained on those latent repr. + noise. The goal of diff. transformer is to learn to remove the noise. -¿ if a model learns how to denoise the latent vector, it learns also how to generate new latent vectors.

- A parametric internal representation models SVG graphics as continuous parameters (e.g., Bezier control points and colors) that can be directly optimized with gradients.

- Coarse-to-fine refinement starts from a rough initial set of shapes and then gradually adds paths, repeatedly checking and adjusting them to better match the target description.

- T2V w/NPR: They trained the Neural Path Representation (dual-branch VAE) on the FIGR-8 SVG dataset and it learns: how vector paths map to a latent path space and how to decode latents back into valid vector paths. VSD extends SDS by projecting diffusion-based pixel gradients into the neural path latent space for more stable vector optimization.

- DeepSVG is used by sampling or modifying latent vectors: each random sample produces a new SVG icon, and by interpolating or editing these latent vectors, users can generate variations or blends of existing icons.

Reproducibility Overview

Model	Code	Pre-trained	Hardware	Cost of time	Reproduction Feasibility
LIVE	Yes	No	RTX 3090	2,600–10,000 s	Feasible on high-end consumer GPU (slow).
SAMVG	No	No	RTX 3090	139–2,000 s	Re-implementation possible but requires effort.
VectorFusion	No	No	Not specified	Unknown	Not reproducible due to lack of public implementation.
SVGDreamer	Yes	No	V100 (≥31GB VRAM)	7–14 min	Requires high-VRAM GPU and LoRA pretraining.
SVGDreamer++	Yes	No	A800 (≥31GB VRAM)	7 min	Requires high-VRAM GPU and LoRA pretraining.
NeuralSVG	Yes	No	Not stated	Unknown	Hardware requirements unclear.
NIVeL	Pending	No	A100	5 min	High-end GPU required.
T2V-NPR	Yes	VAE (unknown)	RTX 3090	13 min	Feasible on high-end consumer GPU.
DeepSVG	Yes	VAE (unknown)	2×1080Ti (training)	Unknown	Feasible on consumer GPU.
DeepIcon	No	Transformer (unknown)	Not specified	–	Not reproducible (no official code).
SVGFusion	No	VP-VAE + VS-DiT (unknown)	Multi-A800	24–36 s	No official implementation available.
Im2Vec	Yes	VAE (unknown)	Not specified	Unknown	Likely feasible on consumer GPU.
LayerTracer	Yes	DiT + LoRA (unknown)	A100 (80GB)	27–34 s	Enterprise GPU required.

Declaration on oath

I hereby certify that I have written my master thesis independently and have not yet submitted it for examination purposes elsewhere. All sources and aids used are listed, literal and meaningful quotations have been marked as such.

March 30, 2026, Your Name

Consent to plagiarism check

I hereby agree that my submitted work may be sent to PlagAware (https://my.plagaware.com/) in digital form for the purpose of checking for plagiarism and that it may be temporarily (max. 5 years) stored in the database maintained by PlagScan as well as personal data which are part of this work may be stored there.

Consent is voluntary. Without this consent, the plagiarism check cannot be prevented by removing all personal data and protecting the copyright requirements. Consent to the storage and use of personal data may be revoked at any time by notifying the faculty.

March 30, 2026, Your Name